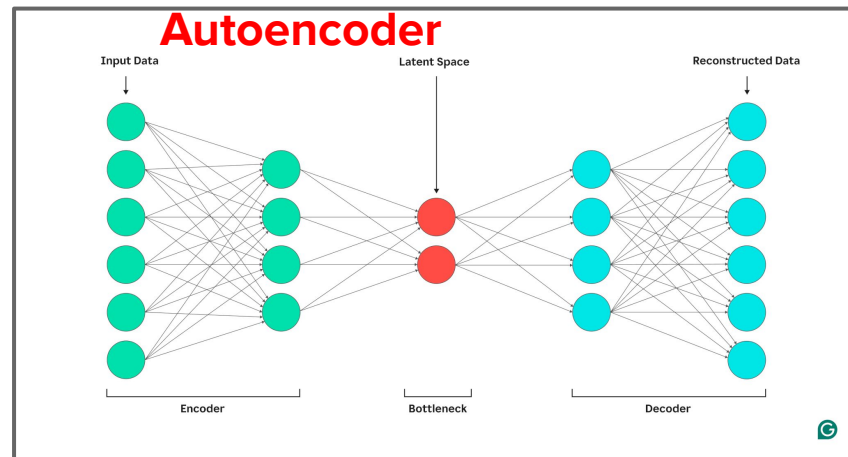
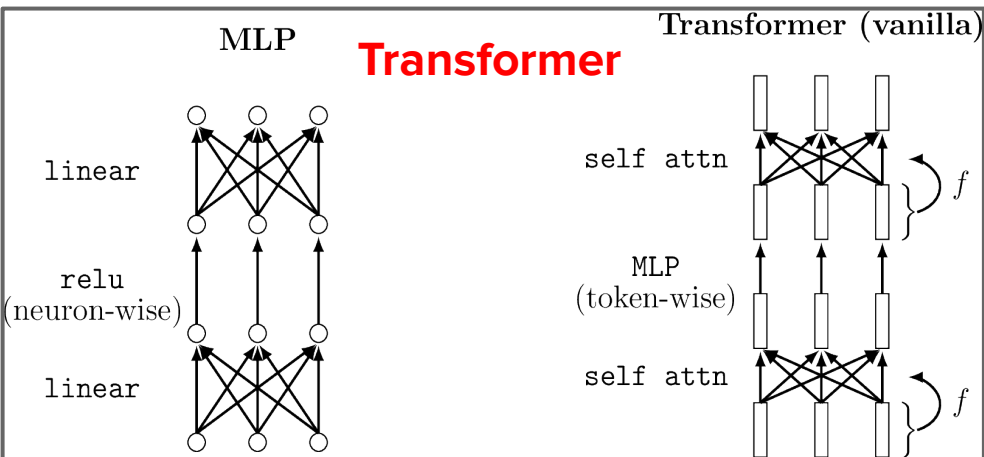
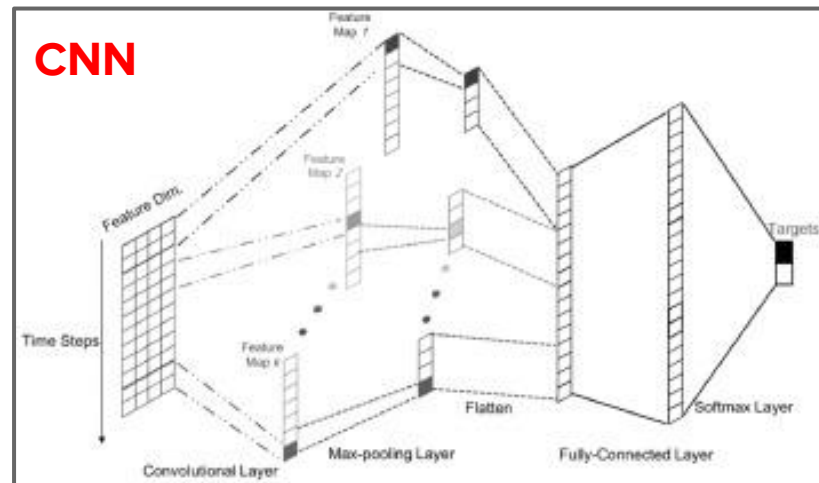
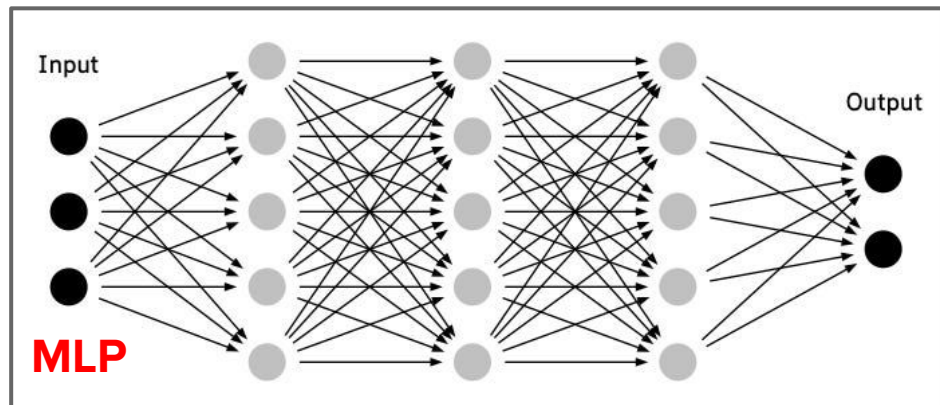


Supervised Learning: Linear and Logistic Regression

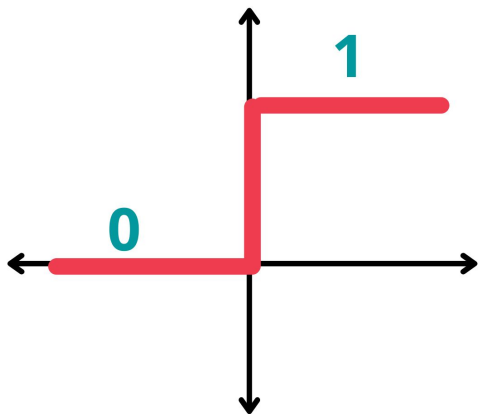


Mahesh Mohan M R
Department of AI
Indian Institute of Technology Kharagpur

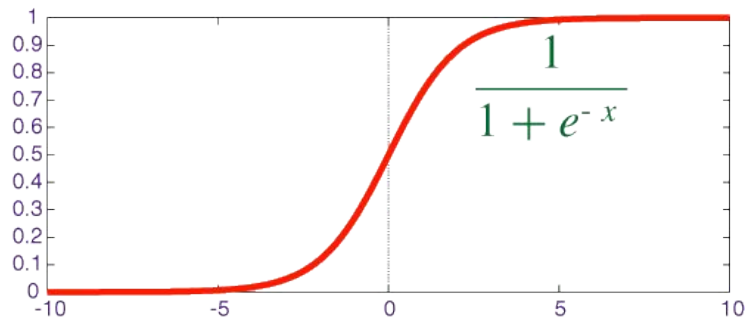
Artificial Neural Network



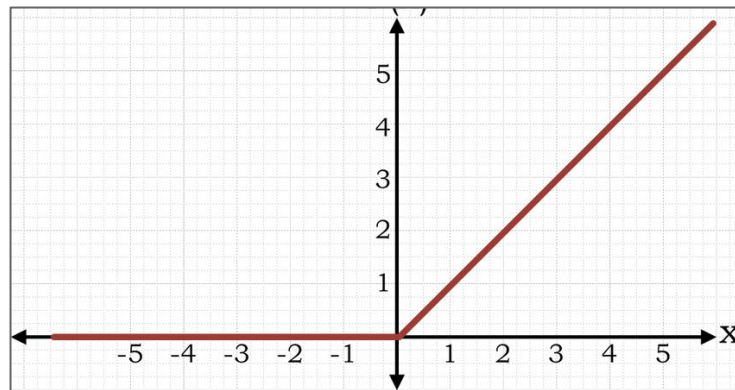
Activations



Binary Step Activation Function

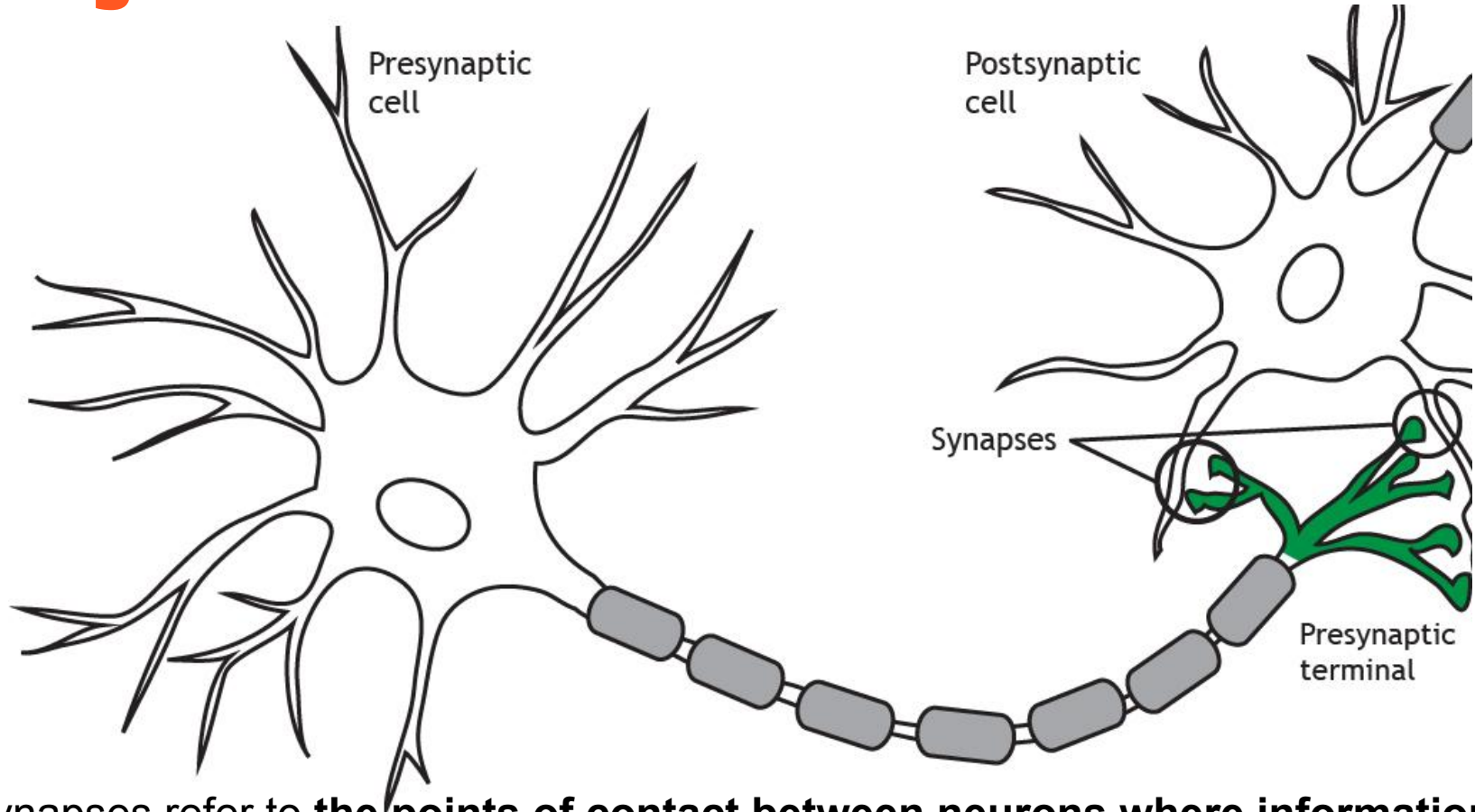


Sigmoid Activation Function



ReLU Activation Function

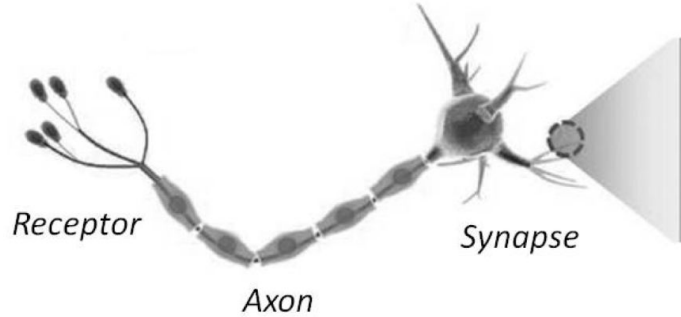
Biological Neuron



Synapses refer to **the points of contact between neurons where information is passed from one neuron to the next.**

Biological Neuron to Neural Network

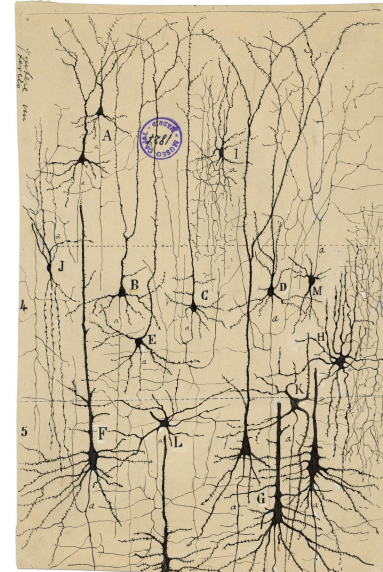
Sensory Neuron



Neural Network

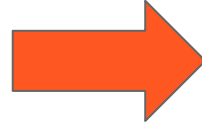
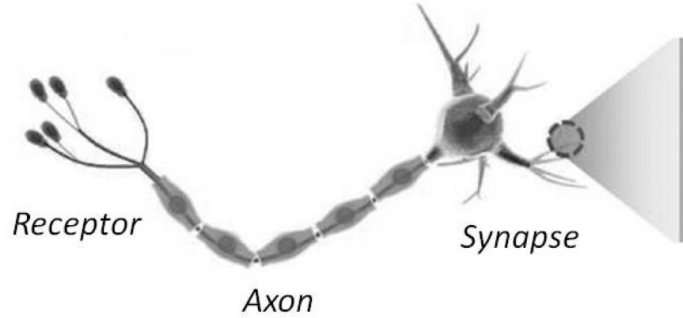


Santiago Ramón y Cajal
(1906 Nobel Laureate)



Artificial Neuron to Neural Network

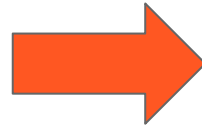
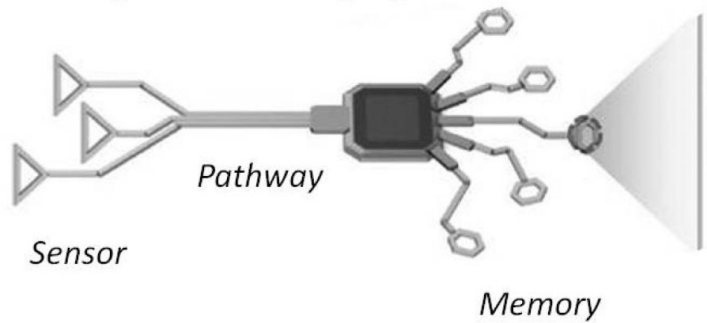
Sensory Neuron



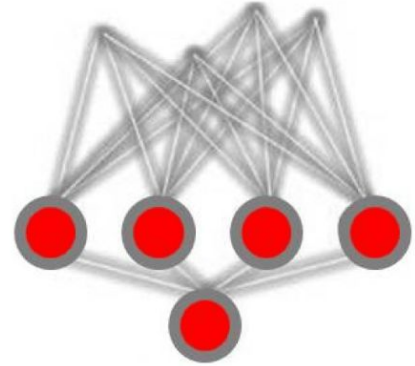
Neural Network



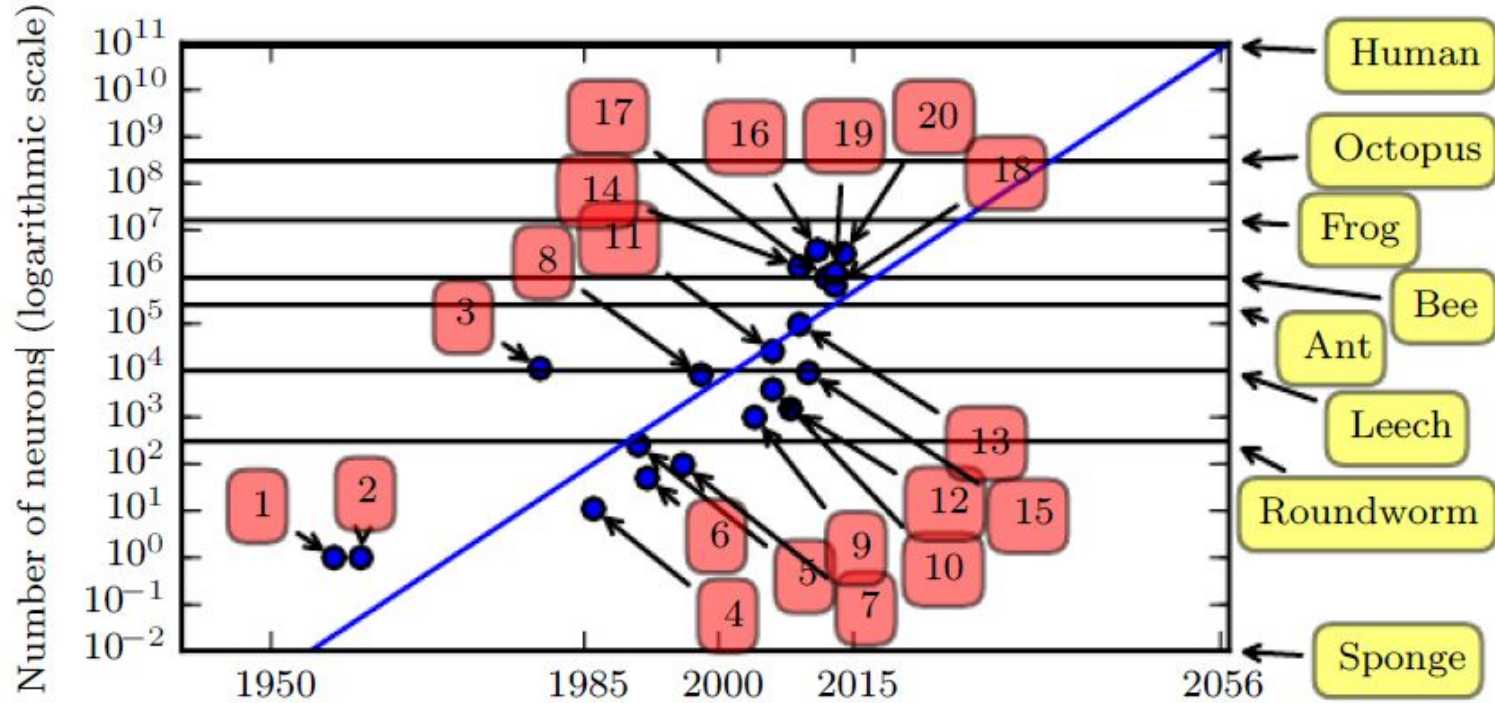
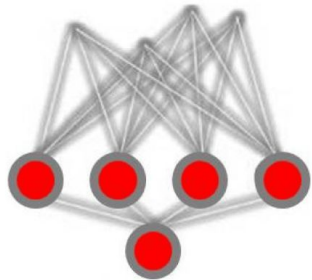
Artificial Sensory System



Artificial Neural Network



Biological Neural Network (NN) to Artificial NN



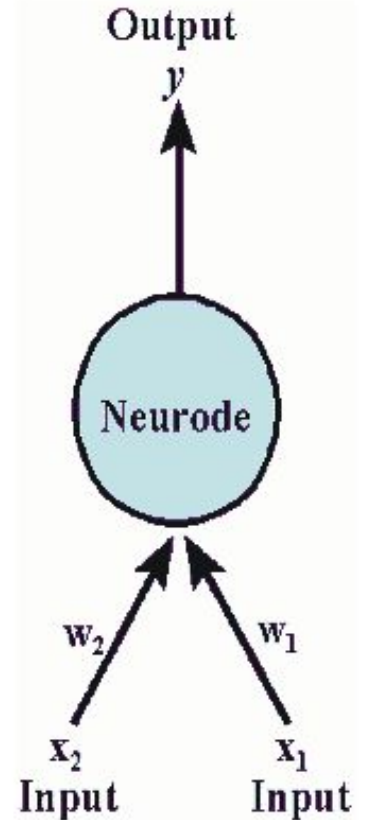
History: McCulloch Pitts Neuron - 1943



Warren
McCulloch

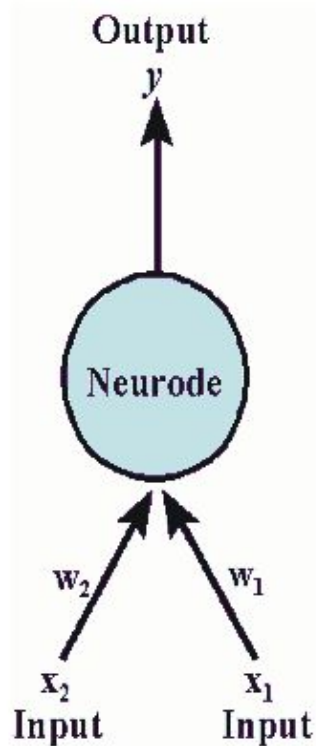


Walter
Pitts



1943 paper, "A Logical Calculus of Ideas Immanent in Nervous Activity"

History: McCulloch Pitts Neuron - 1943

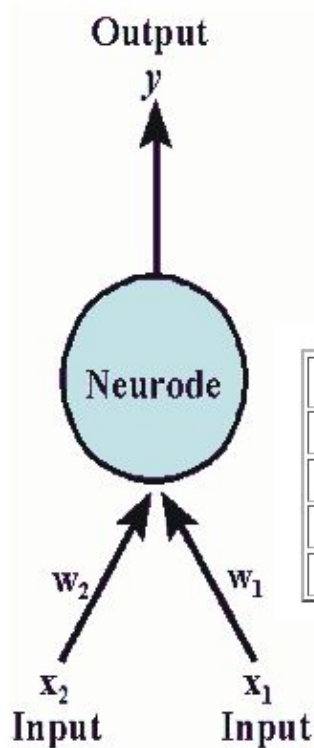


- Inputting either a 1 or 0 for each of the inputs, where 1 represented true and 0 false.
- Weights: Acts as a tag for excitatory (+1) or inhibitory (-1) paths
- The threshold was given a real value, say 1, which would allow for a 0 or 1 output if the threshold was met or exceeded

Input x_1	Input x_2	Output
0	0	0
0	1	0
1	0	0
1	1	1

$$\sum x_i w_i \geq \theta$$

History: McCulloch Pitts Neuron - 1943



- Inputting either a 1 or 0 for each of the inputs, where 1 represented true and 0 false.
- Weights: Acts as a tag for excitatory (+1) or inhibitory (-1) paths
- The threshold was given a real value, say 1, which would allow for a 0 or 1 output if the threshold was met or exceeded

Input x_1	Input x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

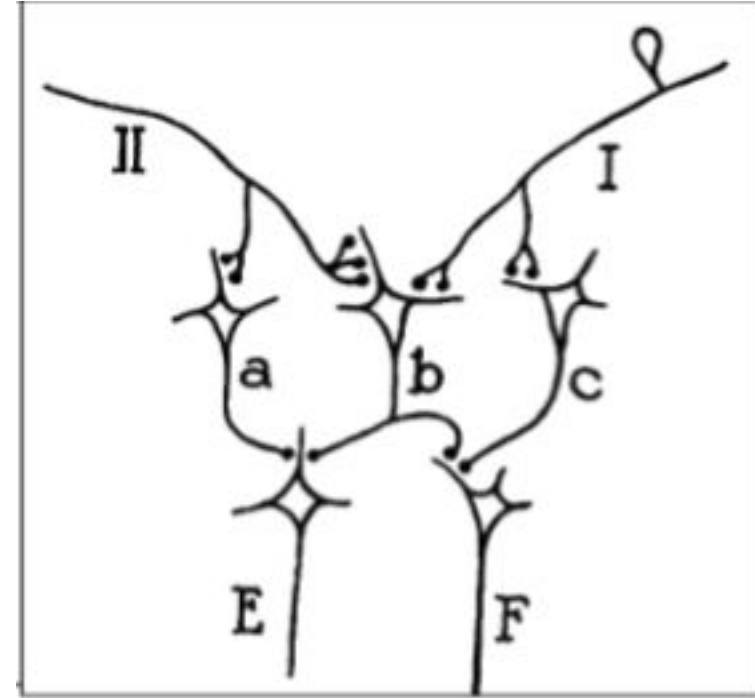
$$\sum x_i w_i \geq \theta$$

History: Hebbian cell - 1949



Research Area: The neurophysiology of learning and memory

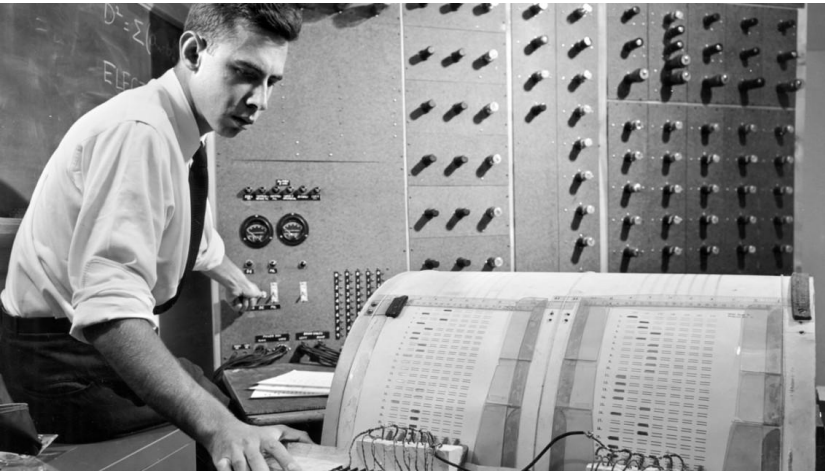
1949 book, "The Organization of Behavior".



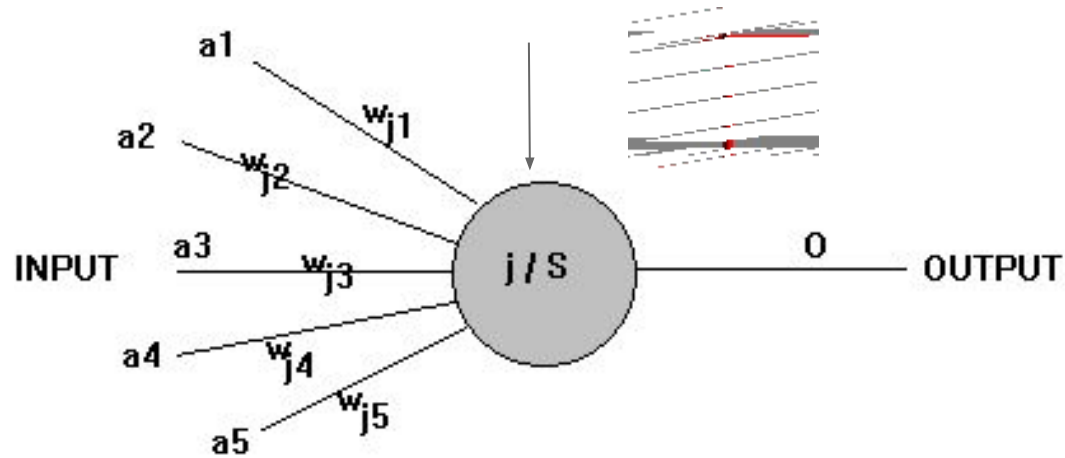
Chemical Neurotransmitters

History: Perceptron - 1958

Frank Rosenblatt, using the McCulloch-Pitts neuron & the findings of Hebb, went on to develop the first perceptron



The first machine “capable of having an original idea.”

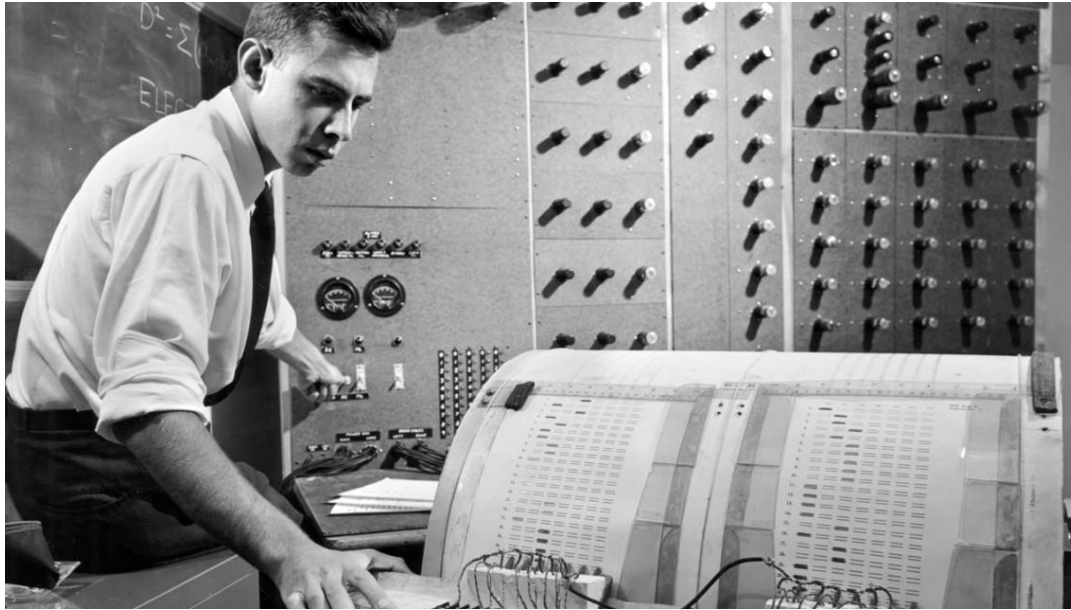


Perceptron

(1958). "The perceptron: A probabilistic model for information storage and organization in the brain"

History: Perceptron - 1958: First AI wave

Frank Rosenblatt, using the McCulloch-Pitts neuron & the findings of Hebb, went on to develop the first perceptron



Given an elementary α -perceptron, a stimulus world W , and any classification $C(W)$ for which a solution exists

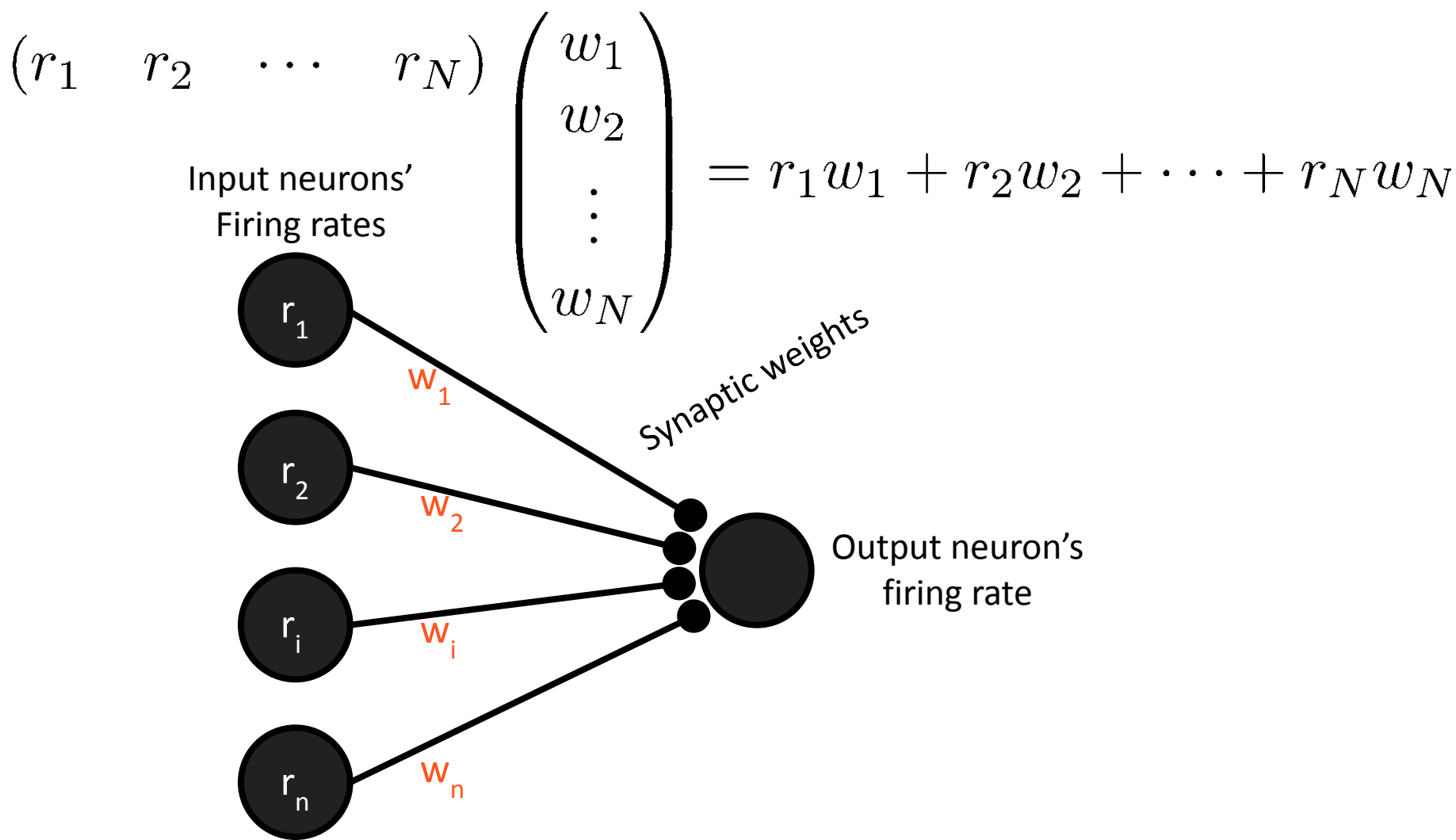
(1958). "The perceptron: A probabilistic model for information storage and organization in the brain"

Multiplication: Dot product (inner product)

$$\vec{x} \cdot \vec{y} =$$
$$(x_1 \quad x_2 \quad \cdots \quad x_N) \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} = x_1 y_1 + x_2 y_2 + \cdots + x_N y_N$$

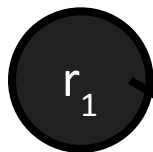


Outer dimensions give
size of resulting matrix





Input neurons'
Firing rates



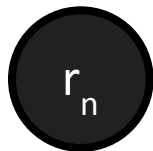
w_1



w_2



w_i



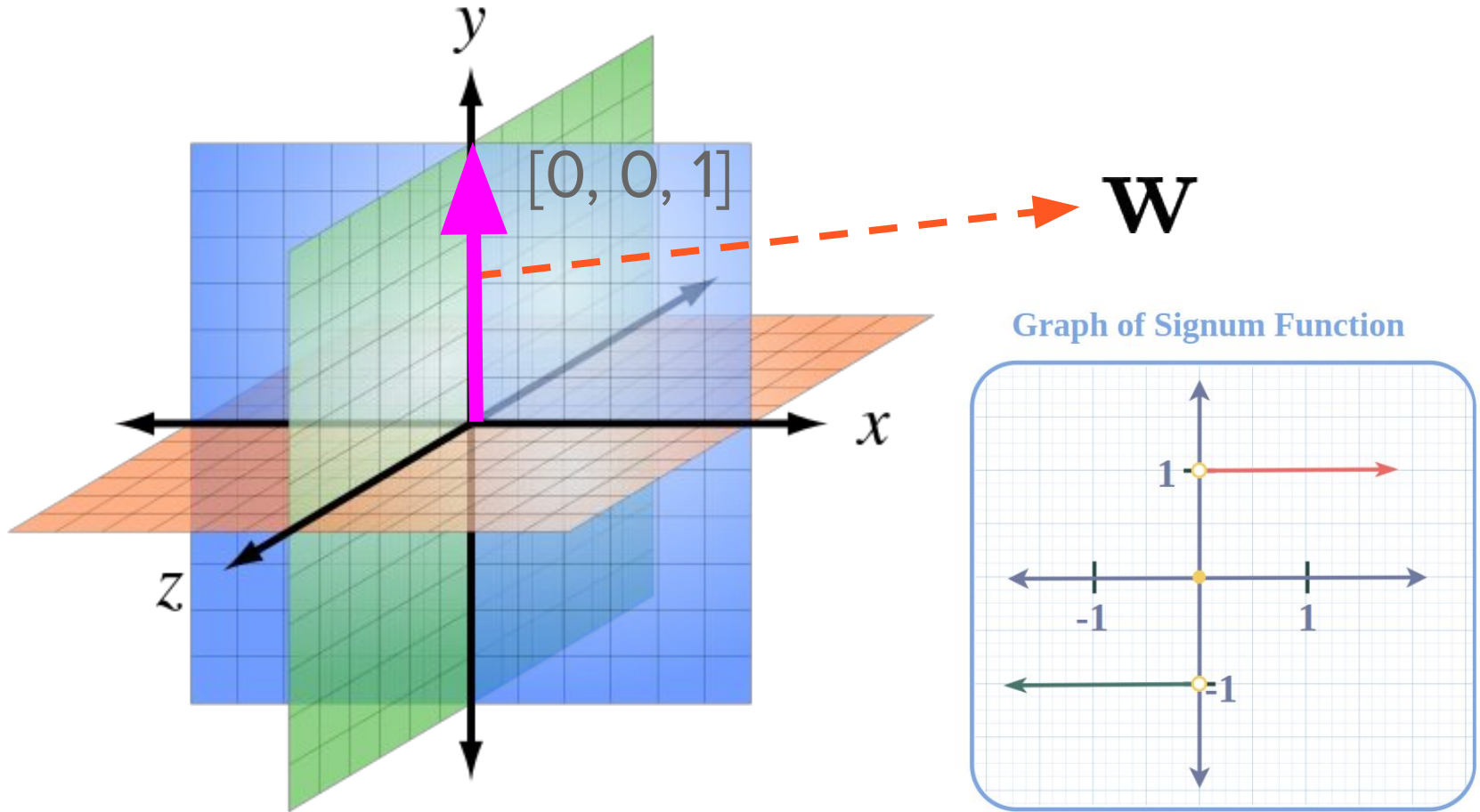
w_n

Synaptic weights

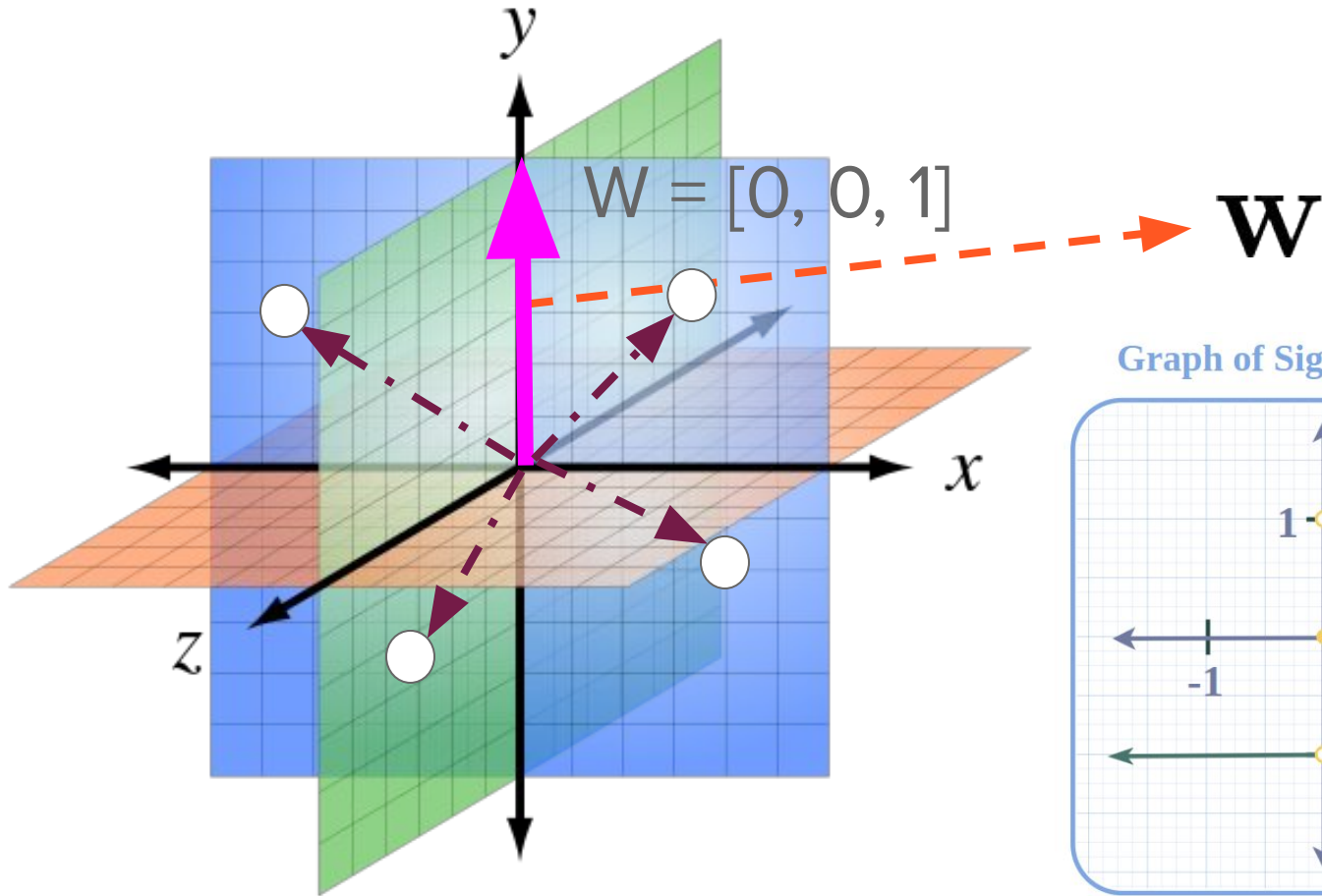
Output neuron's
firing rate

A diagram showing two vectors, $\vec{r}$ and $\vec{w}$, originating from the same point. The angle between them is θ . A red bracket under the horizontal component of $\vec{r}$ is labeled with the equation $\vec{r} \cdot \vec{w} = |\vec{r}| |\vec{w}| \cos(\theta)$. A red dashed line and a right-angle symbol illustrate the projection of $\vec{r}$ onto $\vec{w}$.

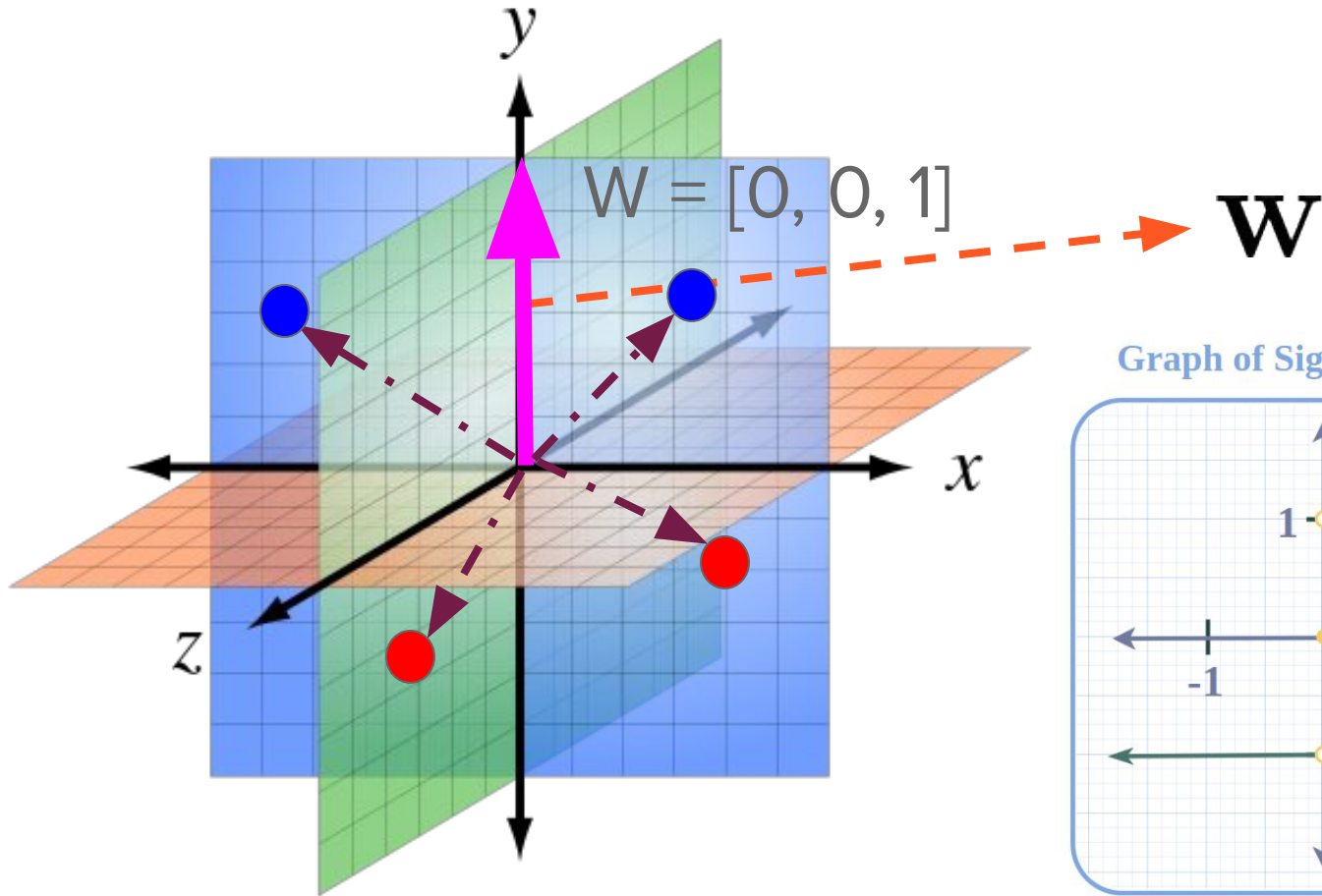
Pre-Inner Product Interpretation



Pre-Inner Product Interpretation



Pre-Inner Product Interpretation



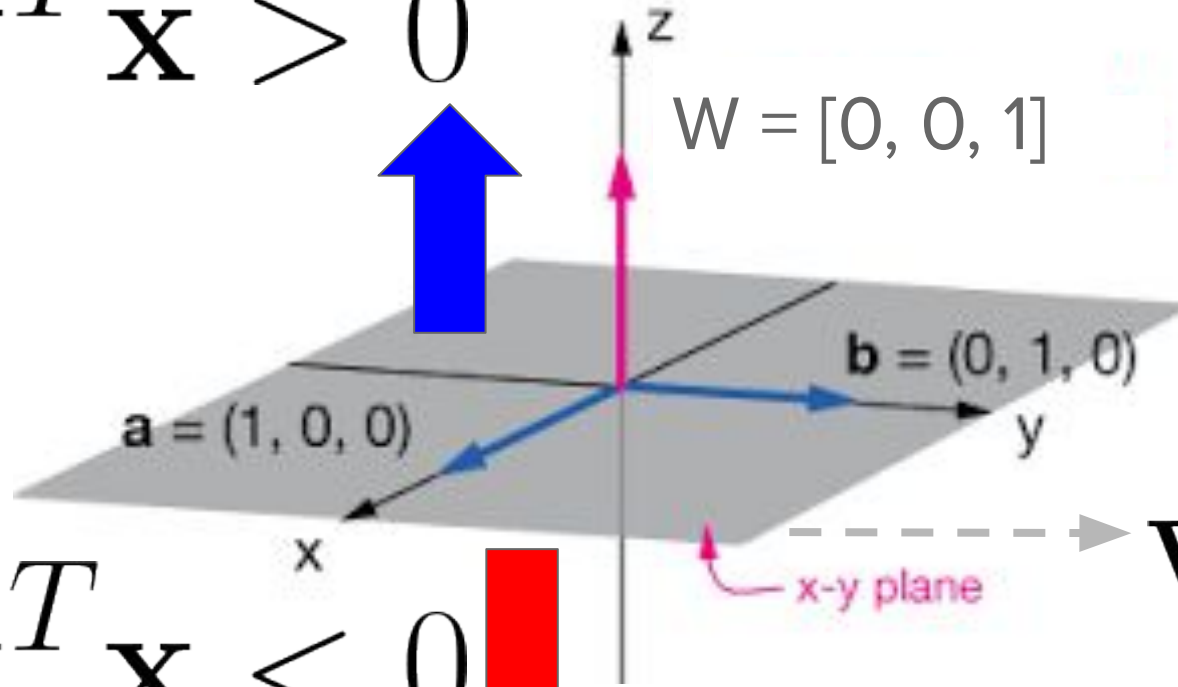
Post-Inner Product Interpretation



$$\mathbf{w}^T \mathbf{x} > 0$$



$$\mathbf{W} = [0, 0, 1]$$

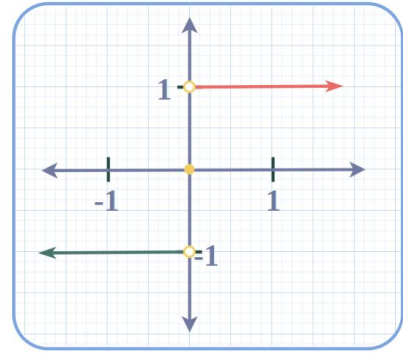


$$\mathbf{w}^T \mathbf{x} < 0$$

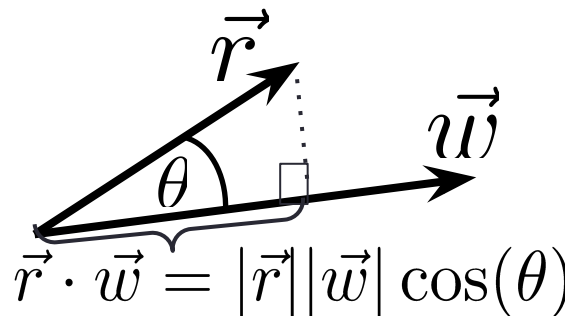


$$\mathbf{w}^T \mathbf{x} = 0$$

Graph of Signum Function

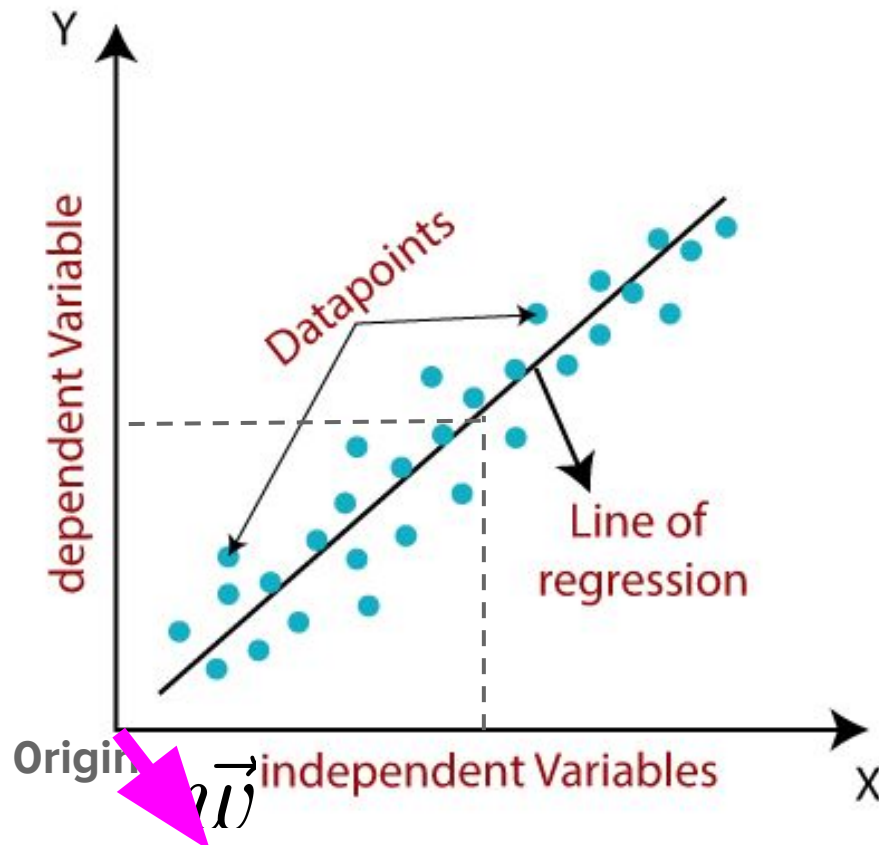
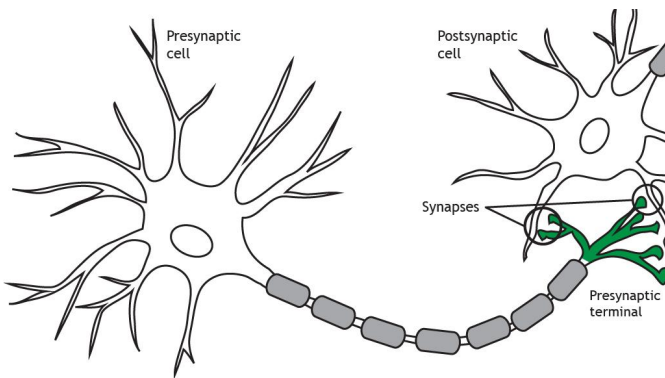


Linear Models for Regression

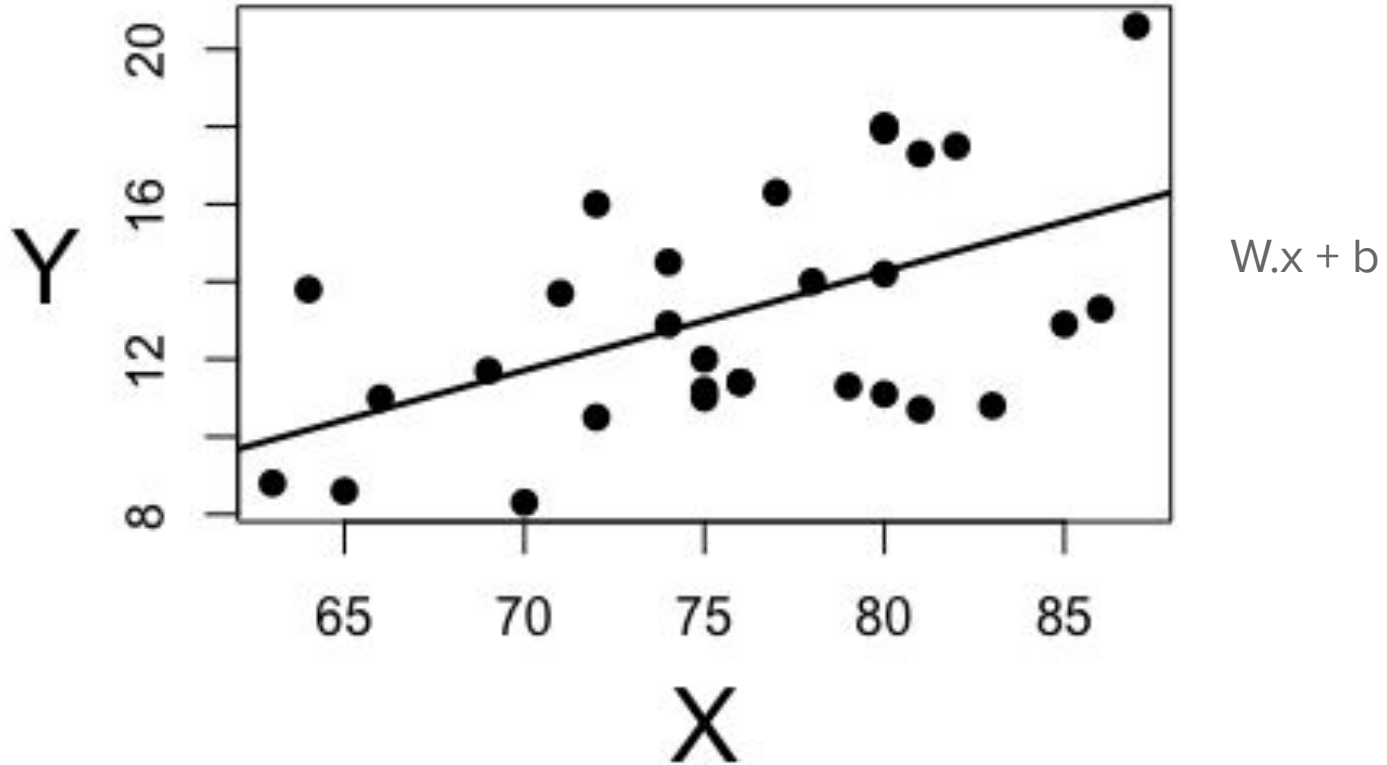


A diagram showing two vectors, $\vec{r}$ and $\vec{w}$, originating from the same point. The angle between them is labeled θ . A dashed line from the tip of $\vec{r}$ perpendicular to $\vec{w}$ illustrates the projection of $\vec{r}$ onto $\vec{w}$. Below the vectors, the equation $\vec{r} \cdot \vec{w} = |\vec{r}| |\vec{w}| \cos(\theta)$ is written.

$$\vec{r} \cdot \vec{w} = |\vec{r}| |\vec{w}| \cos(\theta)$$



Linear Models with Bias



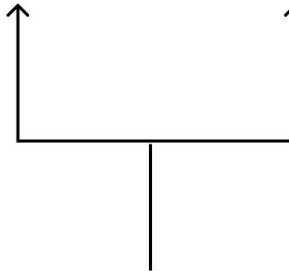
Linear Models with Bias

$$y' = b + w_1 x_1$$

Prediction Bias Weight Feature value

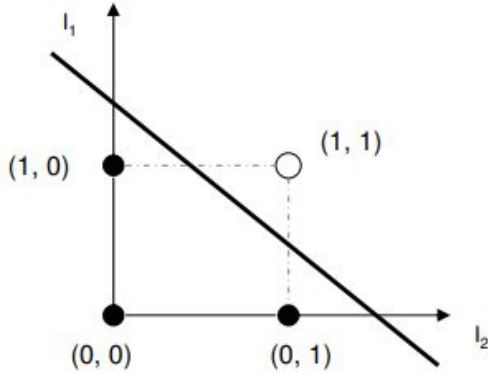


Calculated from training

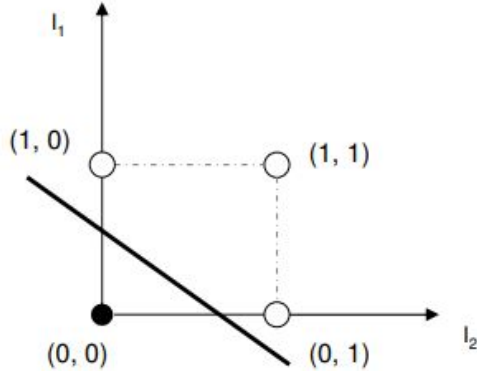


XOR Problem: 1969

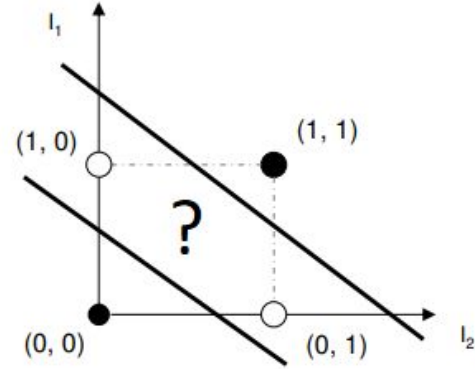
AND		
I_1	I_2	out
0	0	0
0	1	0
1	0	0
1	1	1



OR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	1

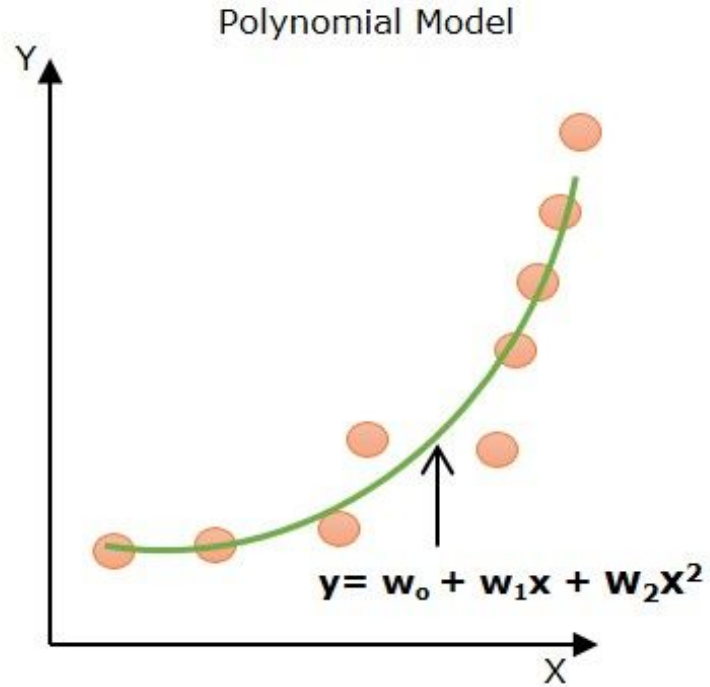
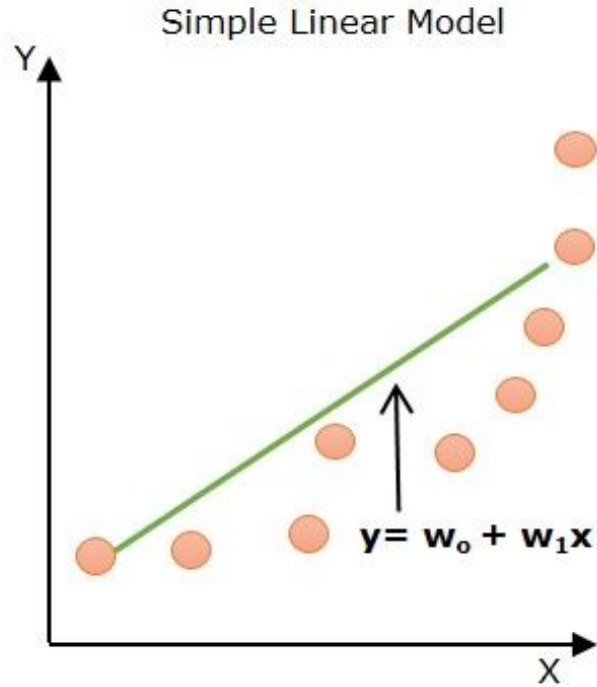


XOR		
I_1	I_2	out
0	0	0
0	1	1
1	0	1
1	1	0



1969, Minsky co-authored with [Seymour Papert](#), *Perceptrons: An Introduction to Computational Geometry*.

Polynomial Regression



Von Neumann's elephant



Fermi



Neumann

With four parameters I can fit an elephant, and with five I can make him wiggle his trunk.

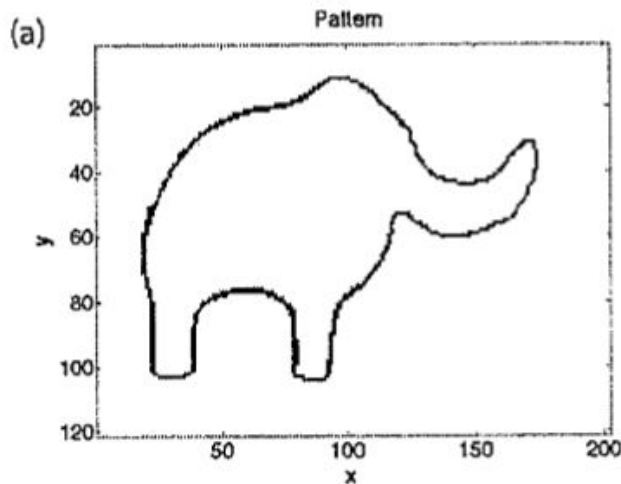
Von Neumann about Fermi's Results

Neumann: How many arbitrary parameters he used for his calculations?

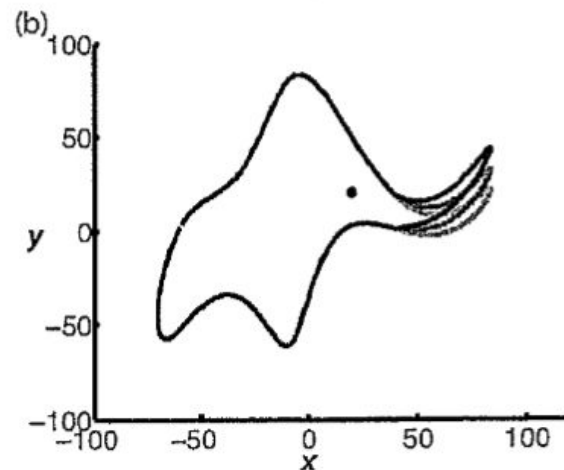
Fermi: Many (with physical interpretation)

Neumann: ***"With four parameters I can fit an elephant, and with five I can make him wiggle his trunk."***

Parameter
$p_1 = 50 - 30i$
$p_2 = 18 + 8i$
$p_3 = 12 - 10i$
$p_4 = -14 - 60i$
$p_5 = 40 + 20i$



Outline of elephant (4 params)



Three snapshots of the wiggling trunk (via 5 th param)

Polynomial Regression

$$\text{L1: } R(\theta) = \|\theta\|_1 = \sum_{i=1}^n |\theta_i|$$

$$\text{L2: } R(\theta) = \|\theta\|_2^2 = \sum_{i=1}^n \theta_i^2$$

Ridge Regression: L2 Norm on Weights

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

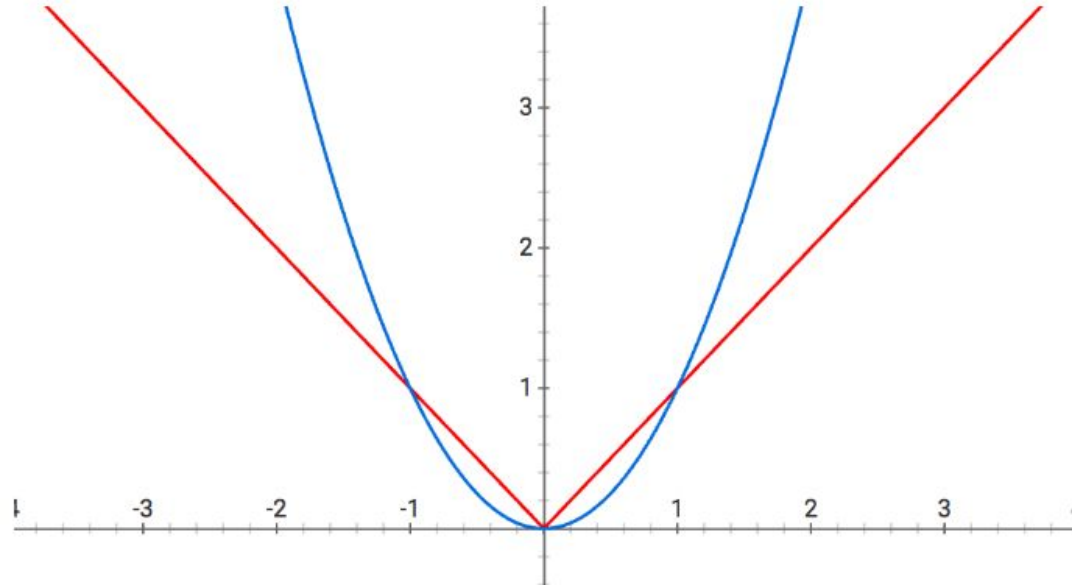
Lasso Regression: L1 Norm on Weights

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i|$$

Note: (a) θ is the bias and coefficients of the polynomial model

(b) α is the regularization weight

Understanding L1 and L2 Penalty

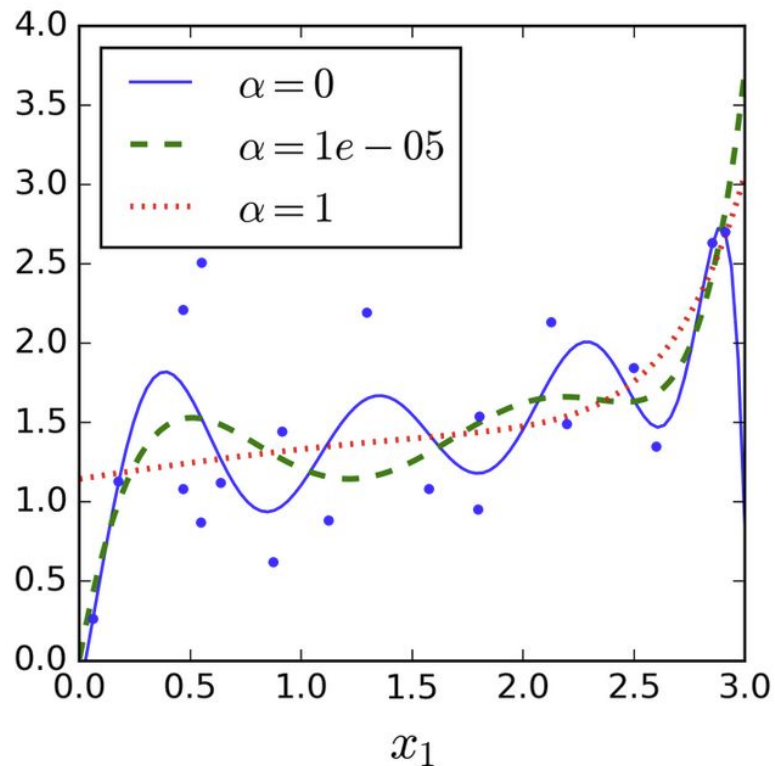


One dimensional case

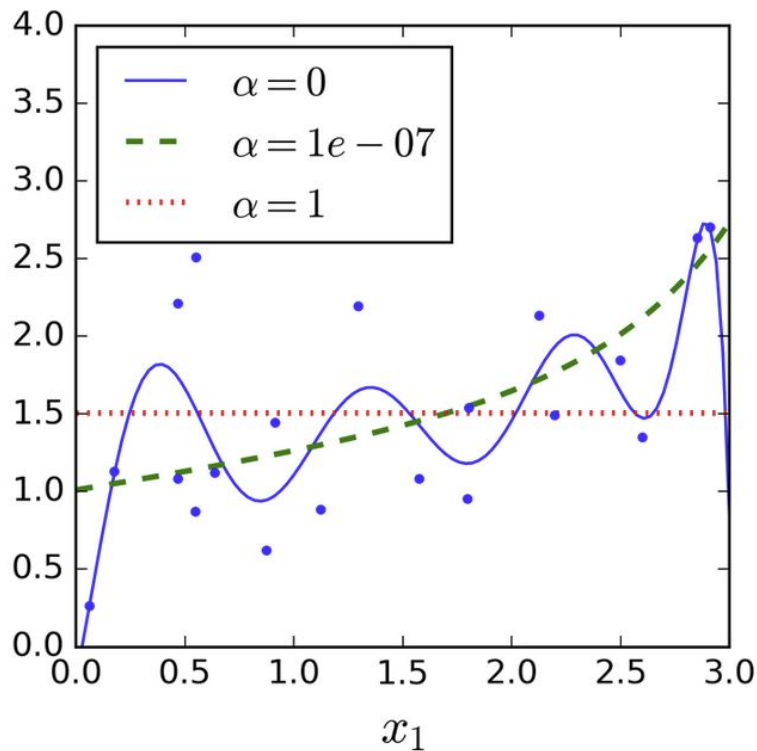
$$\text{L1: } R(\theta) = \|\theta\|_1 = \sum_{i=1}^n |\theta_i|$$

$$\text{L2: } R(\theta) = \|\theta\|_2^2 = \sum_{i=1}^n \theta_i^2$$

Ridge Vs Lasso Regression

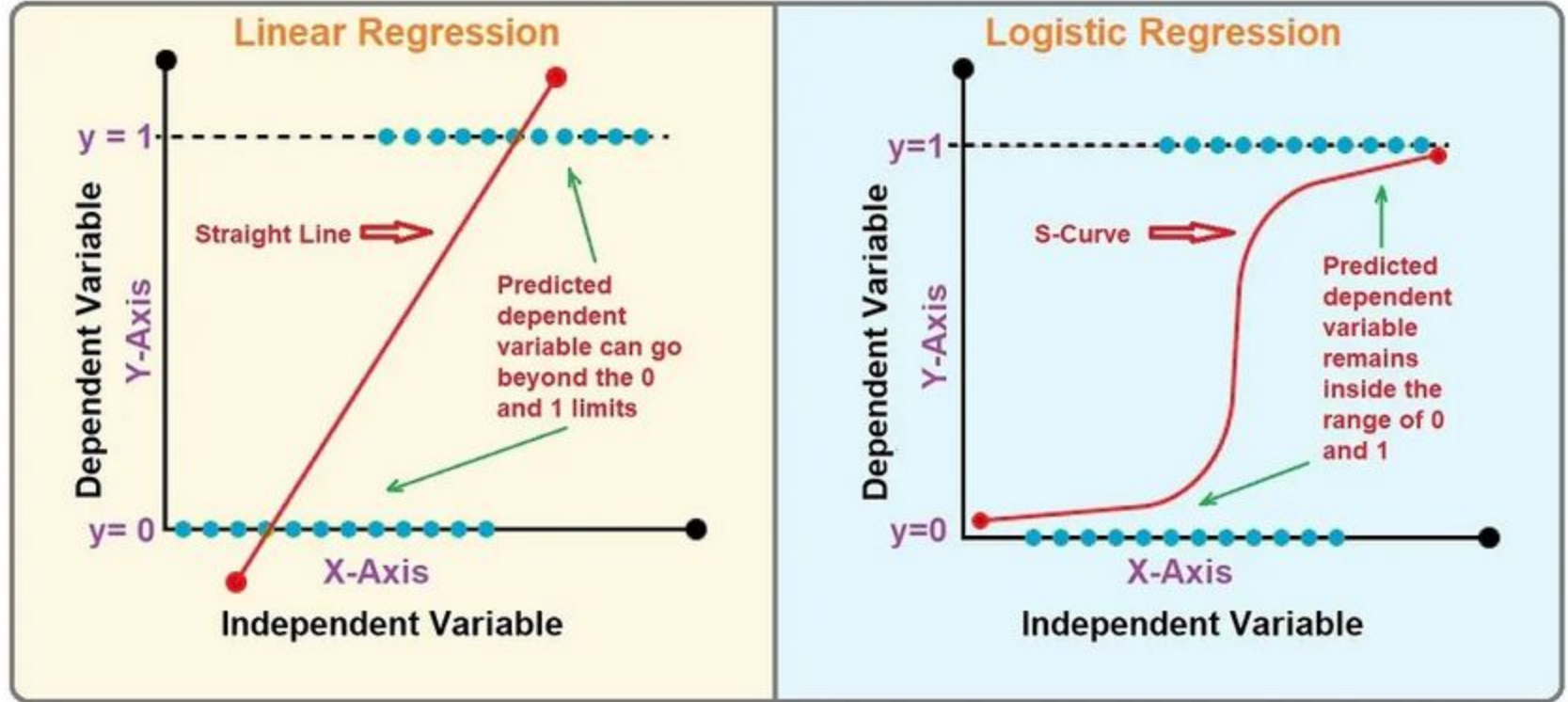


Ridge Regression: L2 Norm on Weights



Lasso Regression: L1 Norm on Weights

Logistic Regression



Logistic Function

- Mathematical form

- $\text{LR}(x) = \frac{e^x}{1+e^x}$

Logistic Function

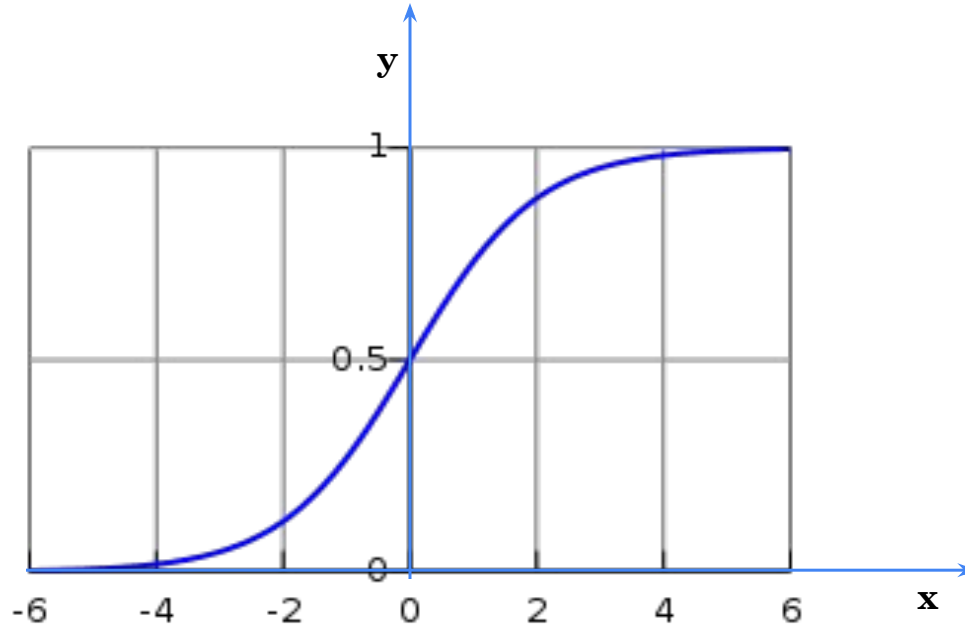
- Mathematical form

- $LR(x) = \frac{e^x}{1+e^x}$

- Properties of Logistic function, $LR(x)$

1. What is the maximum value?
close to 1
2. What is the minimum value?
close to 0
3. When it attains maximum value?
when $x \rightarrow \infty$
4. When it attains minimum value?
when $x \rightarrow -\infty$

Shape of Logistic Function



Other names

- Sigmoid function
- S-curve (because of S-like shape)

Preliminaries

- **Inherently a 2-class classifier**

- The classes traditionally called
 - 0 and 1OR
 - positive and negative

- **Logistic Regression is a probabilistic classifier**

- Given an input feature vector $\mathbf{x}$, it computes the probability that $\mathbf{x}$ belongs to **class 1** (or positive)

$$P(y = 1|\mathbf{x})$$

Preliminaries

"odds" measure the likelihood of an event by comparing the probability of it happening to the probability of it not happening (Odds = $P/(1-P)$)

- Odds

$$\frac{P(y=1|x)}{1-P(y=1|x)}$$

- Properties of odds

1. Always positive
2. Lies between 0 *and* ∞



Logistic Regression Model

- **Main assumption**

- The **logarithm of odds** is a **linear function** of the features of input $\mathbf{x}$

- **Mathematically**

[assume that $\mathbf{x}$ has n features: $\mathbf{x} = (x_1, x_2, \dots, x_n)$]

$$\log \left(\frac{P(y = 1|\mathbf{x})}{1-P(y = 1|\mathbf{x})} \right) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n = Z \text{ (say)}$$

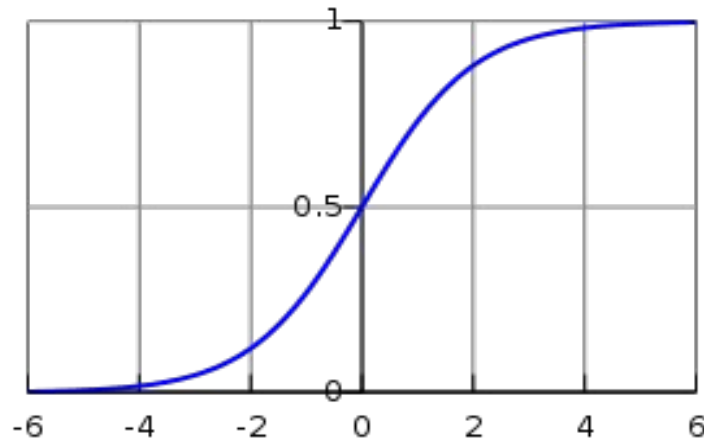
Organizing the above you get

$$P(y = 1|\mathbf{x}) = \frac{e^Z}{1+e^Z}$$

- **For convenience we write, instead**

$$P(y = 1|\mathbf{x}; \theta) = \frac{e^Z}{1+e^Z}$$

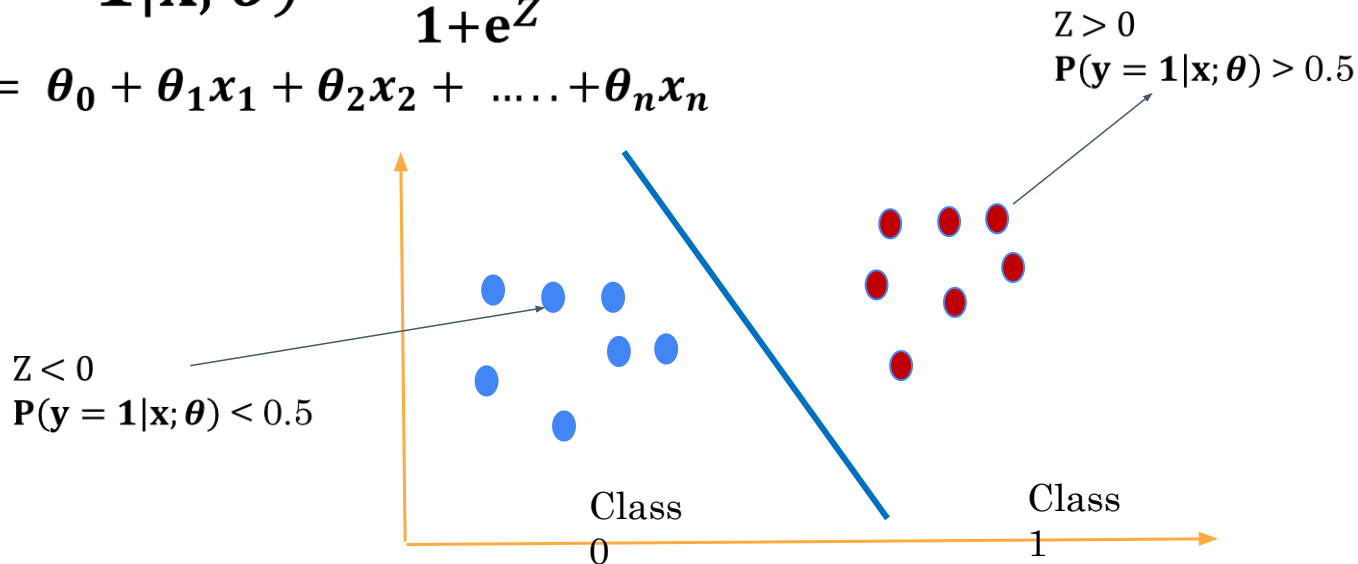
where θ is a short form of model parameters $(\theta_0, \theta_1, \dots, \theta_n)$



What kind of boundary it can model?

- $P(y = 1|x; \theta) = \frac{e^Z}{1+e^Z}$

- $Z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

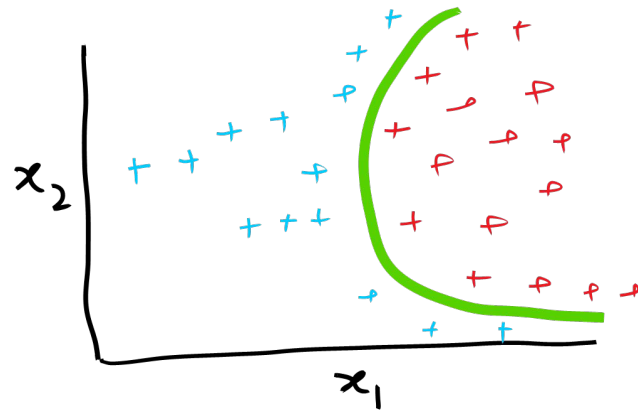


What about non-linear boundary?



Dealing with Non-linear Boundary

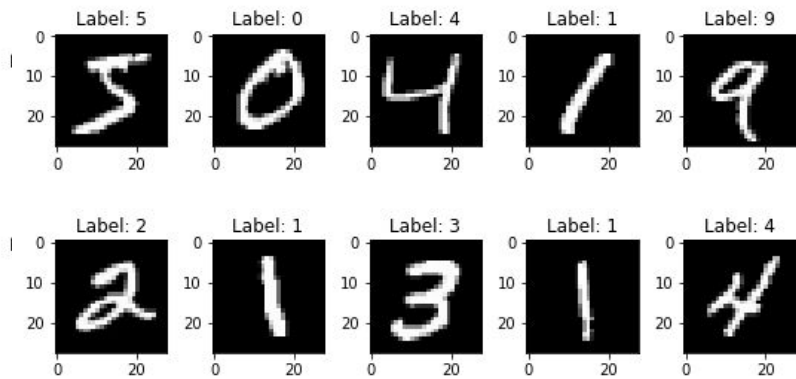
Z determines the boundary



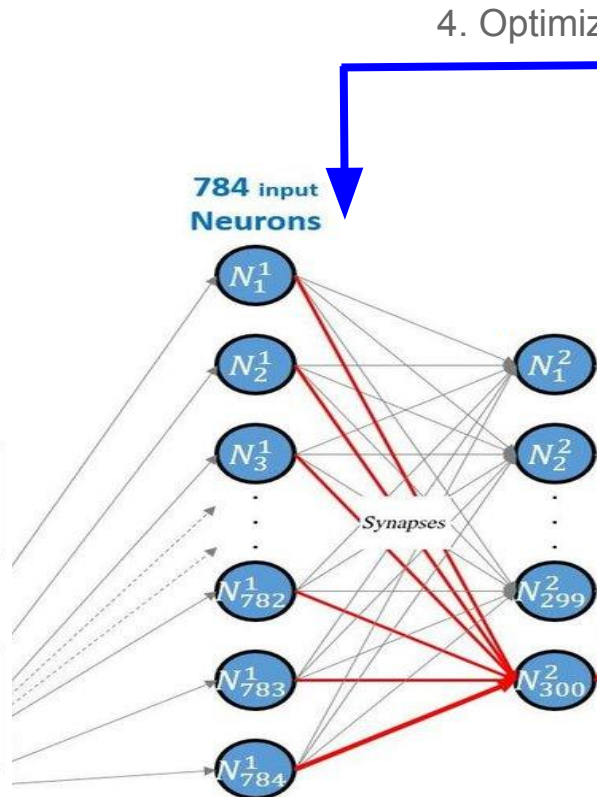
In this case, Z needs to be a quadratic function

Recap: AI: How to Solve it?

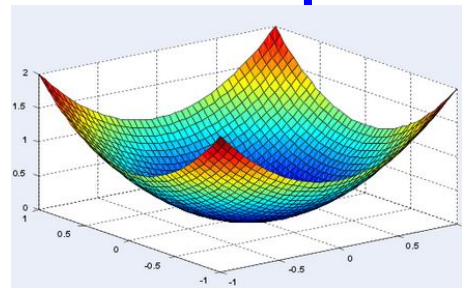
1. Collect Labelled Dataset ✓
2. Design ANN Architecture ✓
3. Define Loss Function ✓
4. Optimize weights



1. Collect Labelled dataset



2. Design Artificial Neural Network



3. Loss Function

4. Optimize Weight

Optimize Weights

Review of Differential Calculus

- Consider the univariate function

$$y = f(x) \quad (\text{for example: } y = x^2)$$

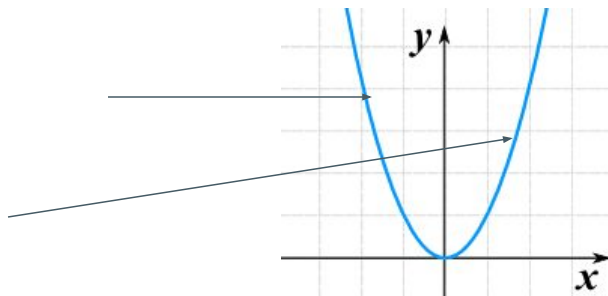
- $\frac{dy}{dx}$ at $x=x_0$ means rate of change of y w.r.t x

- $\frac{dy}{dx} < 0$ means the function is decreasing

- $\frac{dy}{dx} > 0$ means the function is increasing

What is the sign of $\frac{dy}{dx}$
here?

What is the sign of $\frac{dy}{dx}$
here?



Review of Differential Calculus

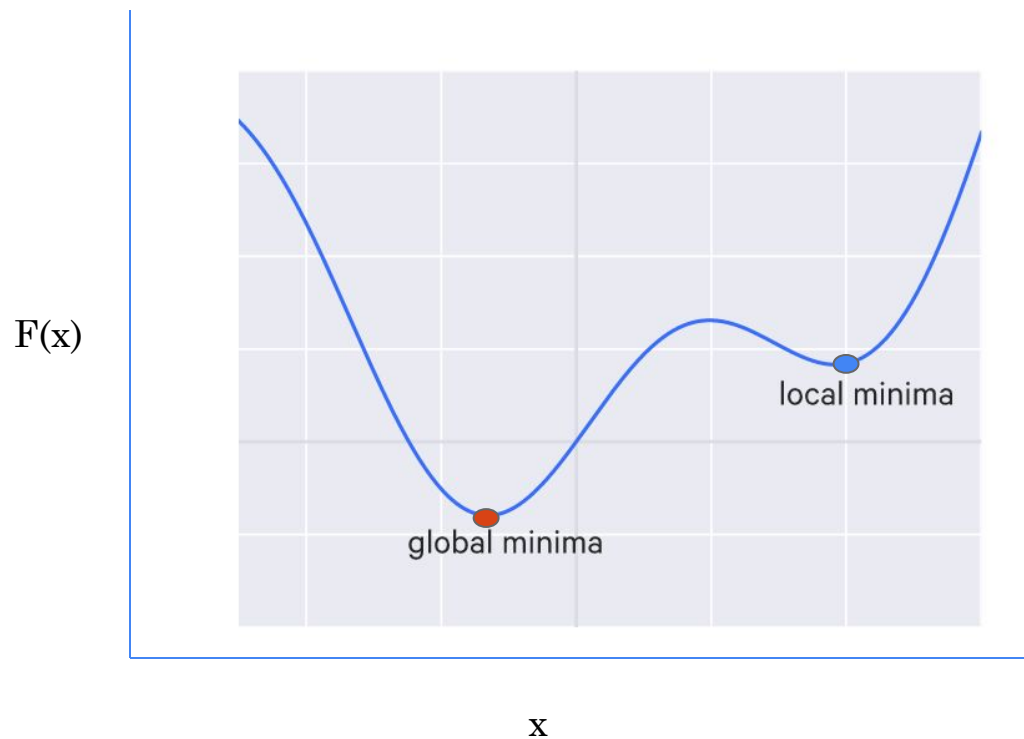
- **Multivariate function**

- Function containing more than one independent variable
- Example: $y = x^2 + z^2 + w^2$

- **Partial differentiation**

- Vary one variable at a time and keep others fixed
- $\frac{\partial y}{\partial x} = 2x, \frac{\partial y}{\partial z} = 2z, \frac{\partial y}{\partial w} = 2w$

Local and Global minima



Optimize Weights via Loss Function

- Goal

Minimize a function w.r.t the parameters that the function contains

- The error/objective function

$$E = \sum_{i=1}^m (y^{(i)} - F(x^{(i)}))^2$$

$F \rightarrow ML\ model$

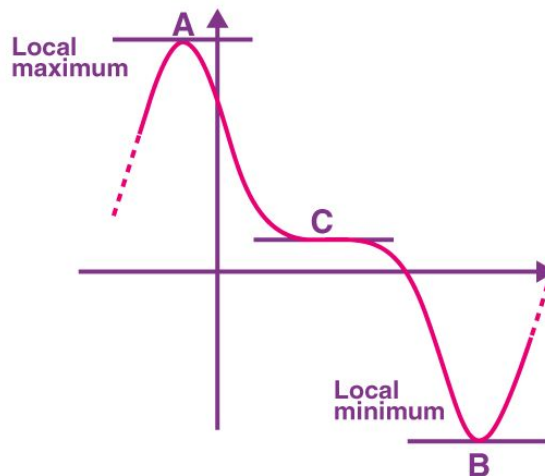
Training data	
Input	Output

- Two objectives

1. Only minimum value of the function
2. Find the value of the parameters that will minimize the function

Recap: Closed Form Expression

	Maximum	Minimum
Necessary condition	$\frac{dy}{dx} = 0$	$\frac{dy}{dx} = 0$
Sufficient condition	$\frac{dy}{dx} = 0 ; \frac{d^2y}{dx^2} < 0$	$\frac{dy}{dx} = 0 ; \frac{d^2y}{dx^2} > 0$



Linear Regression

Labelled Data $\begin{bmatrix} x_{k1} \\ x_{k2} \\ \dots \\ x_{kn} \end{bmatrix} \Rightarrow y_k$

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{d1} & x_{d2} & \dots & x_{dn} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \dots \\ y_d \end{bmatrix}$$

$$\nabla w = (X^T X)w - X^T y$$

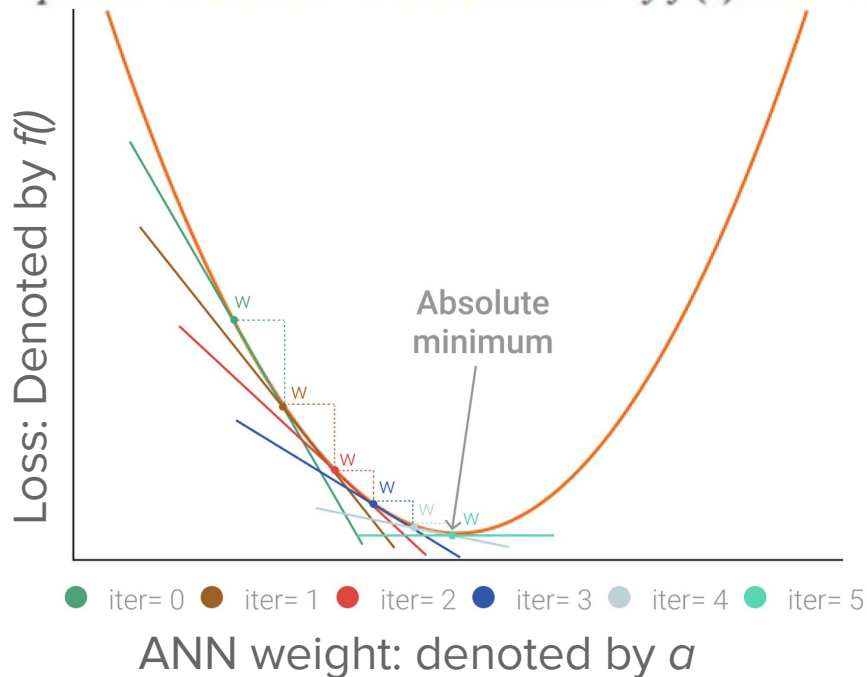
$$w = (X^T X)^{-1} X^T y$$

Recap: Gradient Descent

Definition 1 Suppose f is a real valued function and a is a point in its domain of definition. The derivative of f at a is defined by

$$\lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}$$

provided this limit exists. Derivative of $f(x)$ at a is denoted by $f'(a)$.



What happens to Loss $f(a+h) - f(a)$ if the weight update h is $-\lambda f'(a)$?

Note: $\lambda > 0$

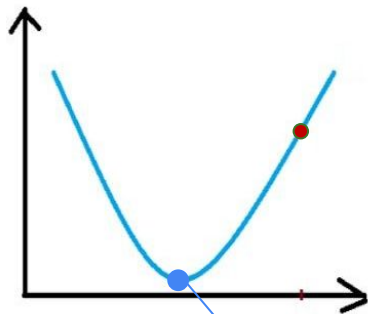
Answer: $f(a+h) - f(a)$ is negative, which means $f(a+h) \leq f(a)$.

Optimizing ANN: Update each ANN weight a as $a - \lambda f'(a)$, where λ is the learning rate.

Gradient Descent: Demonstration

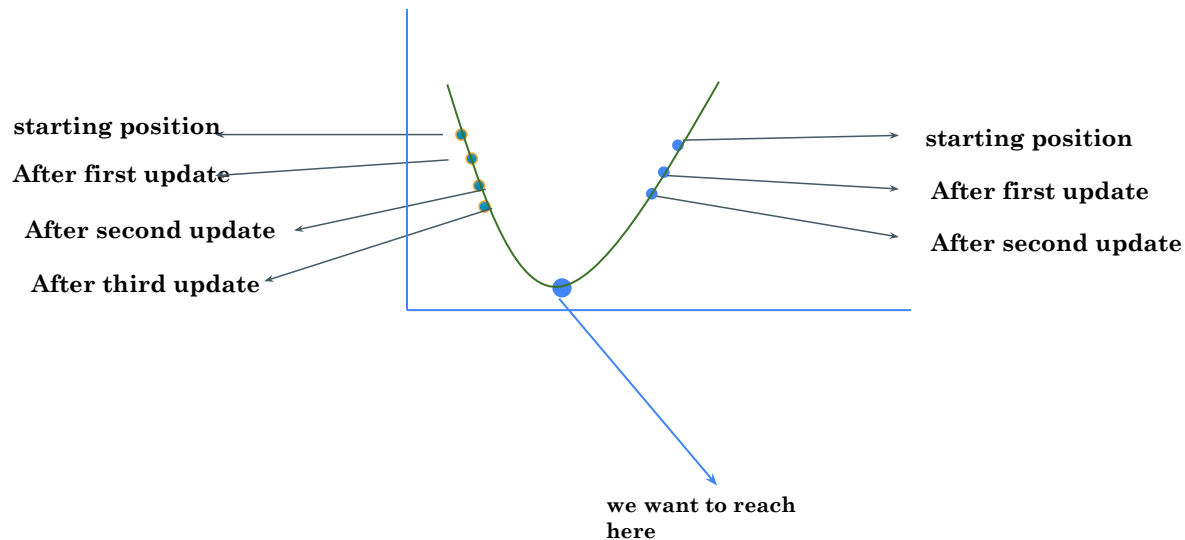
- **Role of gradient descent**

- To find the values of parameters that minimize an objective function



we want to reach here

Gradient Descent: Demonstration

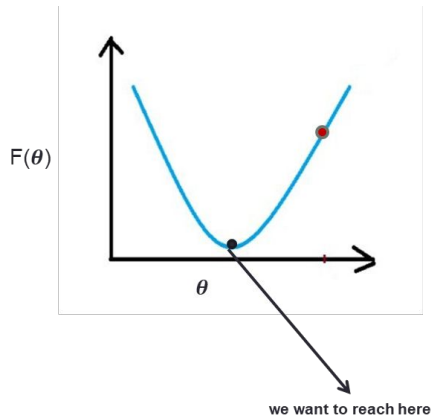


Gradient Descent Algorithm

1. Randomly set the values of parameters (thetas)

2. Repeat until convergence

- $\theta_j^{t+1} = \theta_j^t - r * \frac{\partial E}{\partial \theta_j}$ for all j



- How to check convergence?

- Two widely used approaches

1. Iterate a fixed number of times
2. Stop if parameters do not change much in two consecutive iteration

Gradient Descent for Linear Regression

$$y = F(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- **Error function**

$$E = \sum_{i=1}^m (F(x^{(i)}) - y^{(i)})^2 \quad (\text{m} = \text{no. of training samples})$$

- **Gradient descent update rule**

- Requires **partial differentiation** with respect to all parameters

$$\theta_i^{t+1} = \theta_i^t - r * \frac{\partial E}{\partial \theta_i} \quad \text{for all } i$$

Gradient Descent for Linear Regression

- $E_t = \sum_{i=1}^m (F(x^{(i)}) - y^{(i)})^2$ (m = # training sample)

For example: $F(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

- $\frac{\partial E_t}{\partial \theta_0} = 2 \sum_{i=1}^m (F(x^{(i)}) - y^{(i)})$

- $\frac{\partial E_t}{\partial \theta_1} = 2 \sum_{i=1}^m (F(x^{(i)}) - y^{(i)}) * x_1^i$

- So, in general

- $\frac{\partial E_t}{\partial \theta_j} = 2 \sum_{i=1}^m (F(x^{(i)}) - y^{(i)}) * x_j^i$ (for all j) (assume $x_0^i = 1$)

- Update rule:

- $\theta_j = \theta_j - r * 2 \sum_{i=1}^m (F(x^{(i)}) - y^{(i)}) * x_j^i$ (for all j)

-

Gradient Descent for Logistic Regression

- Take one sample from the training data
 - Let $\mathbf{x}$ be the input sample/feature vector
 - Let $\mathbf{y}$ be its label/class
- Given, the input sample $\mathbf{x}$ and the model parameters θ
 - What is the probability that $\mathbf{x}$ belongs to class 1 ?
 - $P(y = 1|\mathbf{x}; \theta) = LR(\mathbf{x})$
 - What is the probability that $\mathbf{x}$ belongs to class 0 ?
 - $P(y = 0|\mathbf{x}; \theta) = 1 - LR(\mathbf{x})$
- Putting together
$$P(\mathbf{y}|\mathbf{x}; \theta) = (LR(\mathbf{x}))^y \times (1 - LR(\mathbf{x}))^{1-y}$$

Gradient Descent for Logistic Regression

- Likelihood function (We have n training examples)
 - $L(\theta) = p(Y|X; \theta)$, where $Y = (y^1, y^2, \dots, y^n)$ and $X = (x^1, x^2, \dots, x^n)$
- Assume that training examples are independent

- $L(\theta) = p(Y|X; \theta) = \prod_{i=1}^n p(y^i|x^i; \theta)$

$$= \prod_{i=1}^n LR(x^i)^{y^i} \times (1 - LR(x^i))^{1-y^i}$$

Apply: $P(y|x; \theta) = LR(x)^y \times (1 - LR(x))^{1-y}$

Estimating Model Parameters

- We now have the likelihood function:

$$L(\theta) = \prod_{i=1}^n LR(x^i)^{y^i} \times (1 - LR(x^i))^{1-y^i}$$

- What we have to do now?

Maximize the likelihood function: $L(\theta) = \prod_{i=1}^n LR(x^i)^{y^i} \times (1 - LR(x^i))^{1-y^i}$

- But $L(\theta)$ is complex to manage

- Alternative solution

- Maximize $\log L(\theta)$ instead

- So, the final likelihood function is

$$\log L(\theta) = \sum_{i=1}^n y^i \log LR(x^i) + (1 - y^i) \log (1 - LR(x^i))$$

Estimating Model Parameters

- Apply stochastic gradient ascent
 - Updating parameters using one training sample at a time

- Gradient ascent: $\theta_j^{new} = \theta_j^{old} + r \frac{\partial \log(L(\theta))}{\partial \theta_j}$

- We just need to find out $\frac{\partial \log(L(\theta))}{\partial \theta_j}$, for all j

- Likelihood function:

$$\log L(\theta) = \sum_i y^i \log LR(x^i) + (1 - y^i) \log(1 - LR(x^i))$$

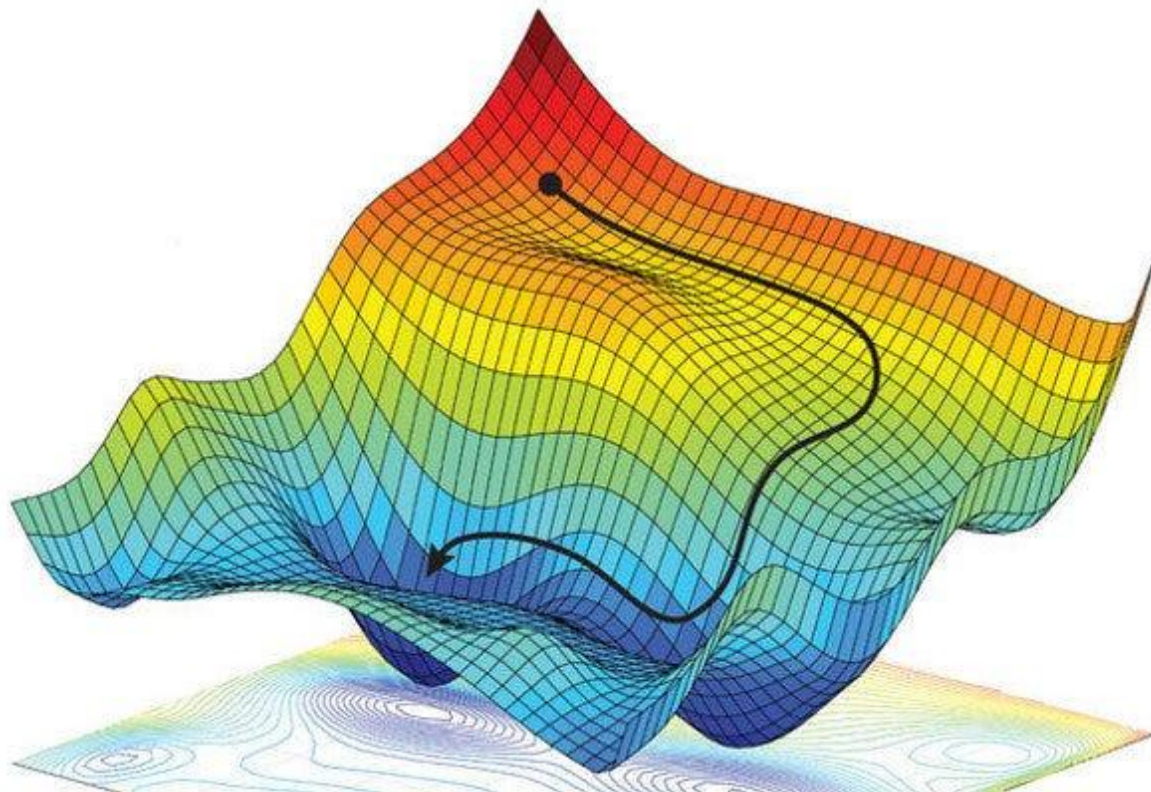
- $\frac{\partial \log(L(\theta))}{\partial \theta_j} = \left(y^i - LR(x^i) \right) * x_j^i$

x_j^i is the j-th feature of i-th training sample

Limitations of Batch Gradient Descent



Loss Calculated using **All** training dataset

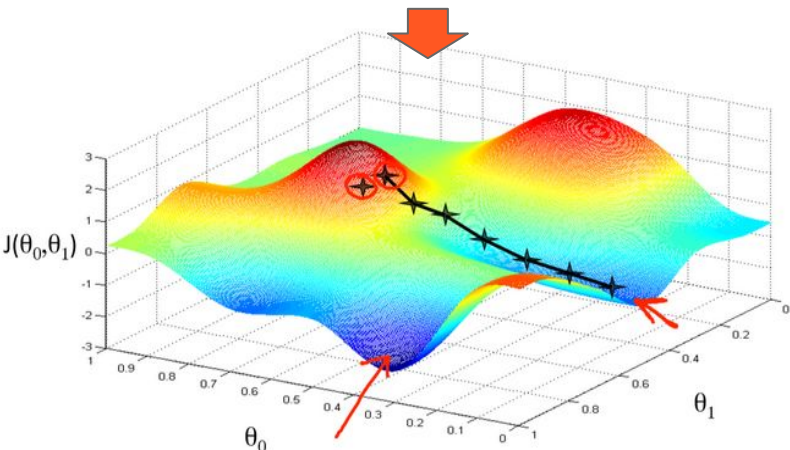
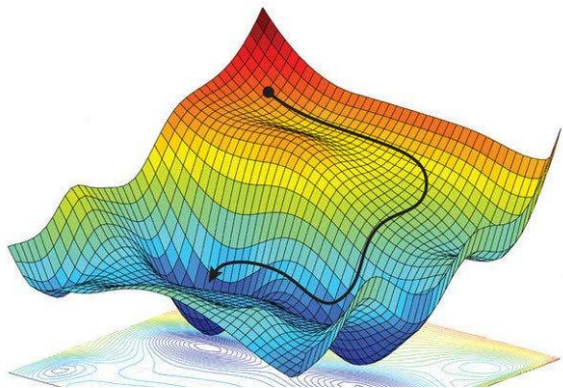


1. Prone to Overfitting
2. Not GPU-friendly
3. May get stuck in local Minima

Stochastic Gradient Descent



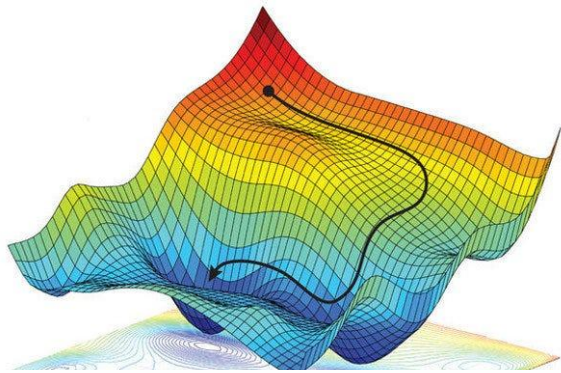
Loss Calculated using **One** training data sample



1. Loss curve may not represent ideal loss (Batch GD) → final solution may be poor.
2. Under use of GPU
3. Hard to converge (as weight updates happen for each data)

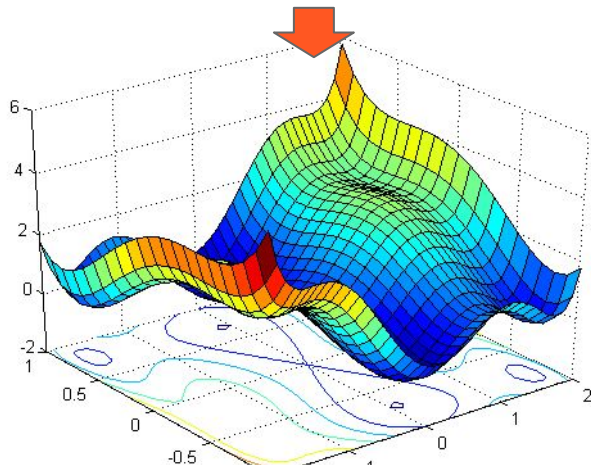
Solution: Mini-batch SGD

Loss Calculated using **A Batch** of training data samples



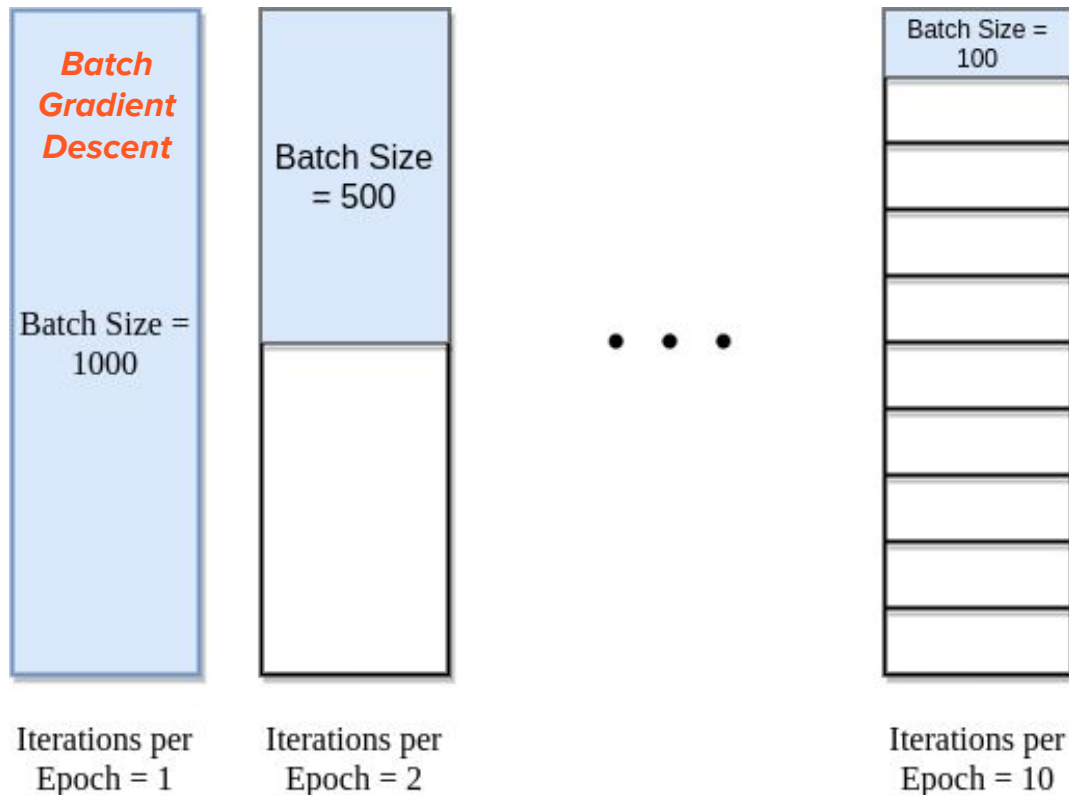
Drawback of SGD

1. Loss curve may not represent ideal loss (Batch GD) → final solution may be poor. — **Better**
2. Under use of GPU — **Better**
3. Hard to converge (as weight updates happen for each data) — **Better**



Mini-batch SGD: Epochs and Iterations

Loss Calculated using **A Batch** of training data samples



Summary of Gradient Descent

- **Batch Gradient Descent**

- Gradient is computed using all training samples

- **Stochastic Gradient Descent**

- Gradient is computed using only one training sample at a time

- **Mini-batch Gradient Descent**

- Gradient is computed on a small batch (say 100) of training samples

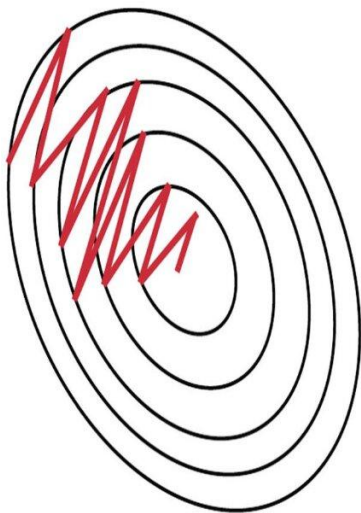


Mini-batch SGD + Momentum,

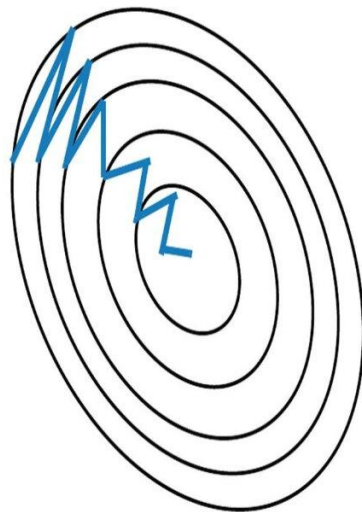
Loss Calculated using **A Batch** of training data samples

Optimizing ANN: Update each ANN weight a as $a - \lambda f'(a)$, where λ is the learning rate.

At t^{th} iter, $v_t = \beta v_{t-1} + 1 - \beta f'(a)$ combines past gradients. Update a as $a - \lambda v_t$.



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

